

A Project Report

On

AI Driven Lora WAN Enabled Solar Powered Multi Sensor Framework for Proactive Flood Prediction and Early Warning

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology

in

**COMPUTER SCIENCE AND ENGINEERING
(WITH SPL IN IOT & AUTOMATION)**

Submitted by

G.V.MAHIDHAR REDDY	224E1A4644
S.MAHAMMAD SALEEM	224E1A4638
R.NIKHITHA SHARMA	224E1A4624
T.DHANUSH	234E5A4601
B.SUMANTH KUMAR	224E1A4635

Under the esteemed guidance of

K.Purnima, MTech., [Ph.D].,

Associate Professor

Dept. of CSE



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SIDDARTHA INSTITUTE OF SCIENCE AND TECHNOLOGY (AUTONOMOUS)

(Approved by A.I.C.T.E., New Delhi Affiliated to J.N.T.U. Anantapur , Ananthapuramu)

Siddharth Nagar, Narayanavanam Road, Puttur– 517 583

2025-2026

SIDDARTHA INSTITUTE OF SCIENCE AND TECHNOLOGY (AUTONOMOUS)

(Approved by A.I.C.T.E., New Delhi Affiliated to J.N.T.U. Anantapur, Ananthapuramu.)
Siddharth Nagar, Narayanavanam Road, Puttur-517583

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(WITH SPL IN IOT & AUTOMATION)



Certificate

This is to certify that the Project entitled
**“AI Driven Lora WAN Enabled Solar Powered Multi Sensor Framework for
Proactive Flood Prediction and Early Warning “**
that is being submitted by

G.V.MAHIDHAR REDDY	224E1A4644
S.MAHAMMAD SALEEM	224E1A4638
R.NIKHITHA SHARMA	224E1A4624
T.DHANUSH	234E5A4601
B.SUMANTH KUMAR	224E1A4635

in partial fulfillment of the requirements for the award of **BACHELOR OF TECHNOLOGY** in **Computer Science and Engineering (WITH SPL IN IOT & AUTOMATION)** to **JNTUA, Ananthapuramu**. This project work or part thereof has not been submitted to any other University or Institute for the award of any degree.

Guide

Head of the Department

Submitted for the University Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

Acknowledgement

An endeavor of a long period can be successful only with the advice of many well-wishers. We take this opportunity to express our deep gratitude and appreciation to all those who encouraged us for successful completion of the project work.

We wish to express my sincere thanks to **Dr. K. ASHOK RAJU., Ph.D., Chairman** of Siddhartha Institute of Science and Technology, Puttur, for providing ample facilities to complete the project work.

Our special thanks to **Dr. M. JANARDHANA RAJU, ME., Ph.D., Principal**, Siddhartha Institute of Science and Technology, Puttur, for his wonderful support during the progress of project work.

We are very much thankful to **Dr. M. A. MANIVASAGAM, M.E, Ph.D., Professor and Head of the Department**, Computer Science and Engineering, Siddhartha Institute of Science and Technology, Puttur, for his valuable suggestions in completion of the project work.

We owe our deep gratitude to our project guide **K.PURNIMA, MTech., [Ph.D.], Associate Professor** Department of Computer Science and Engineering, Siddhartha Institute of Science and Technology, Puttur., who took keen interest on our project work and guided us all along, till the completion of our project work by providing all the necessary information for developing a good system.

Finally, we would like to express sincere thanks to our project coordinator **K PURNIMA, MTech., [Ph.D.], Associate Professor** and faculty member of computer science department, one and all who have helped us to complete the project work successfully.

As a gesture of respect towards our **Family Members, friends** and their support we wholeheartedly dedicate this work.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	i
	LIST OF FIGURES	ii&iii
	LIST OF TABLES	iv
1.	INTRODUCTION	
	1.1 Domain Description	5
	1.2 About the Project	5
2.	LITERATURE SURVEY	6
3.	SYSTEM ANALYSIS	
	3.1 Problem Statement	12
	3.2 Problem Description	12
	3.3 Existing System	13
	3.4 Disadvantages of Existing System	15
	3.5 Proposed System	15
	3.6 Advantages of Proposed System	18
4.	SYSTEM REQUIREMENTS	
	4.1 Hardware Requirements	19
	4.2 Software Requirements	19
	4.3 Feasibility Study	19

Chapter No.	Title	Page No.
5.	SYSTEM DEVELOPMENT	
	5.1 SYSTEM DESIGN	
	5.1.1 System Architecture	22
	5.1.2 Modules	
	5.1.2.1 Arduino Uno	25
	5.1.2.2 Node MCU	26
	5.1.2.3 Sensors	27
	5.1.2.4 solar power	33
	5.1.2.5 Buzzer	35
	5.1.3 Algorithm	35
	5.1.4 UML Diagrams	
	5.1.4.1 Introduction to UML	36
	5.1.4.2 Class diagram	37
	5.1.4.3 Use case Diagram	39
	5.1.4.4 Sequence Diagram	40
	5.1.4.5 Activity Diagram	42
	5.1.4.6 Deployment Diagram	44
	5.2 SYSTEM IMPLEMENTATION	
	5.2.1 Software description	46
	5.2.2 Arduino ide	48
	5.2.3 Hardware description	49
	5.2.4 Database description	51
	5.2.5 Sample code	52
	5.3 SYSTEM TESTING	
	5.3.1 System testing	56
	5.3.2 Types of Testing	60
	5.3.2.1 Unit testing	60
	5.3.2.2 Integration testing	60
	5.3.2.3 Acceptance testing	61
	5.3.3 Test cases	61

Chapter No.	Title	Page No.
6.	RESULTS	
	6.1 Execution procedure	64
	6.2 Screenshots	65
7.	CONCLUSION AND FUTURE ENHANCEMENT	
	7.1 Conclusion	70
	7.2 Future Enhancement	70
	REFERENCES	71

ABSTRACT

The AI-Driven Lora WAN-Enabled Solar-Powered Multi-Sensor Framework for Proactive Flood Prediction and Early Warning is an intelligent environmental monitoring system designed to predict and alert about potential flood events in advance. The system integrates embedded technology, IoT, and artificial intelligence to ensure reliable, real-time monitoring of critical environmental parameters. The Arduino UNO acts as the main controller, interfacing with various sensors such as the rain sensor (for rainfall detection), ultrasonic sensor (to measure rising water levels), pressure sensor BMP180 (for atmospheric pressure), flow sensor (to detect water flow rate), and DHT11 sensor (for temperature and humidity). The data gathered from these sensors are transmitted through LoRa SX1278 modules, enabling long-range, low-power communication even in remote areas. The NodeMCU uploads real-time data to an IoT platform for cloud storage and analytics, while the GPS provides live location and the GSM module sends alert messages to concerned authorities during abnormal conditions. A buzzer and LED indicators provide on-site warnings, and an LCD displays the current sensor status. The system is powered by a solar panel and battery combination, ensuring continuous operation during power outages. Using Python based AI algorithms, such as machine learning classification models, the system predicts possible flood occurrences based on sensor trends. This smart, energy-efficient, and scalable solution provides early warnings to minimize flood damage and improve disaster preparedness in vulnerable regions.

Keywords: Internet of Things, sensors, flood analysis, message alerting, solar power, lcd display.

LIST OF FIGURES

Figure No.	Figure Name	Page No.
5.1	System Architecture	24
5.2	Block Diagram	25
5.3	Arduino	26
5.4	Node MCU	26
5.5	Raindrop sensor	27
5.6	Ultrasonic sensor	28
5.7	Pressure sensor	28
5.8	Dht11 sensor	29
5.9	Flow sensor	31
5.10	Lora sx1278	33
5.11	GSM	31
5.12	GPU	32
5.13	Lora sx1278	33
5.14	Solar power	34
5.15	buzzer	35
5.16	Class diagram	38
5.17	Use case diagram	40
5.18	Sequence diagram	42
5.19	Activity diagram	44
5.20	Deployment diagram	46
5.21	Creating new channel	57
5.22	providing the data	57
5.23	Saving the content	58
5.24	click on api key tab&code for Node mcu	59
5.25	Uploading data into thing speak	60

LIST OF FIGURES

Figure No.	Figure Name	Page No.
6.1	sending message	65
6.2	High temperature detected	66
6.3	Didplaying latitude and longitude	67
6.4	visualizing of real time Location using thing speak	68
6.5	output screenshots	69

LIST OF TABLES

Table No.	Table Name	Page No.
5.1	Flood Detection & Alert Generation	61
5.2	Flood Detection and Warning	62
5.3	Buzzer Functionality	63

CHAPTER 1

INTRODUCTION

1.1 DOMAIN DESCRIPTION

This project falls under the domain of Embedded Systems integrated with Python-based Artificial Intelligence, focusing on smart flood prediction and early warning systems. The domain combines hardware-based real-time sensing with intelligent software-based data analysis to address critical challenges in disaster management. Floods are unpredictable natural disasters that require continuous monitoring and fast decision-making, which can be effectively achieved using embedded and AI-driven solutions.

1.1.1 Importance of Embedded systems in disaster monitoring

Embedded systems are specialized computing units designed to perform dedicated function with high reliability and low power consumption. In disaster monitoring applications, embedded systems enable continuous data acquisition, real-time processing, and autonomous operation. Their compact size, energy efficiency, and ability to interface with multiple sensors make them ideal for deployment in remote and flood-prone areas. This project utilizes embedded technology to monitor environmental conditions without human intervention, ensuring accurate data collection.

1.1.2 Sensor integration in embedded domain

The embedded system integrates multiple sensors to capture critical flood-related parameters. Rain sensors detect rainfall intensity, ultrasonic sensors measure rising water levels, pressure sensors (BMP180) monitor atmospheric pressure variations, flow sensors measure water flow rate, and DHT11 sensors track temperature and humidity. By combining data from multiple sensors, the system improves accuracy and reliability in flood detection. Embedded controllers process sensor inputs in real time, enabling immediate response to abnormal conditions. significance of light-based technologies, including IoT, in addressing global challenges and improving quality of life.

1.1.3 Role of microcontrollers and IoT Hardware

The Arduino UNO acts as the primary embedded controller responsible for sensor interfacing and data preprocessing. NodeMCU enhances the system by providing Wi-

Fi connectivity and IoT capabilities for cloud communication. LoRa SX1278 modules enable long-range, low- power wireless data transmission, which is essential for monitoring large geographical areas. Additional embedded components such as GPS, GSM modules, LCD displays, buzzers, and LEDs support location tracking, alert generation, and local status indication.

Microcontrollers (MCUs):

- Tiny computers that control and interact with hardware components
- Process data from sensors, make decisions, and control actuators
- Examples: Arduino, ESP32, STM32

IoT Hardware:

- Physical components that connect devices to the internet or networks
- Includes sensors, actuators, communication modules (Wi-Fi, Bluetooth, cellular), and more
- Examples: sensors (temperature, humidity), smart devices (lights, thermostats)

1.1.4 Communication technologies in Embedded IoT Systems

Reliable communication is a critical aspect of flood monitoring systems. This project uses LoRa WAN technology, which is widely adopted in embedded IoT applications due to its long communication range, low power consumption, and high reliability. LoRa WAN allows sensor data to be transmitted efficiently even in regions with limited network infrastructure. GSM technology is used as a backup communication channel to send SMS alerts to authorities during emergency conditions, ensuring timely warnings.

1.1.5 Power Management and Solar Energy integration

Power efficiency is a major concern in embedded systems deployed in disaster-prone areas. The proposed framework incorporates a solar-powered energy system with battery backup, enabling uninterrupted operation during power failures and extreme weather conditions. Efficient power management ensures long-term deployment, reduces maintenance costs, and enhances system reliability, making the solution suitable for continuous environmental monitoring .

Solar Panel: Charges a battery during the day

Battery: Powers the system (sensors, MCU, communication) at night or during low light

Power Management: Ensures efficient energy use, prevents overcharging, and protects components.

1.6 Role of python in data analysis and AI

Python forms the intelligence layer of the system by enabling data processing, analysis, and prediction. Sensor data collected from embedded devices is transmitted to the cloud and analyzed using Python-based machine learning algorithms. Classification models identify patterns and trends associated with flood conditions. Python's extensive libraries for data science and machine learning make it suitable for building predictive models that enhance the accuracy and effectiveness of early warning system

1.1.7 Domain Relevance and Real-World Applications

The integration of embedded systems and Python-based AI creates a smart, scalable, and energy-efficient flood prediction framework. This domain is highly relevant to real-world applications such as smart cities, environmental monitoring, disaster management, and climate resilience systems. By combining real-time sensing, long-range communication, renewable energy, and intelligent analytics, the project demonstrates the practical importance of embedded systems and Python in solving critical societal challenges.

1.2 ABOUT THE PROJECT

The AI Driven LoRa WAN Enabled Solar Powered Multi Sensor Framework for Proactive Flood Prediction and Early Warning is a smart environmental monitoring system designed to detect, predict, and alert about potential flood conditions in advance. The project aims to reduce flood-related damage by providing early warnings using real-time data, long-range communication, and artificial intelligence . The system integrates multiple sensors to continuously monitor critical environmental parameters such as rainfall, water level, water flow rate, atmospheric pressure, temperature, and humidity. An Arduino UNO acts as the main embedded controller, collecting and processing sensor data. The collected information is transmitted using LoRa SX1278 modules, enabling long-distance, low-power communication suitable for remote and flood-prone regions. A NodeMCU module uploads real-time sensor data to an IoT cloud platform for storage and monitoring. The system is powered by a solar panel with battery backup, ensuring uninterrupted operation during power failures and extreme weather conditions. For emergency situations, a GSM module sends alert messages to concerned authorities, while a buzzer, LED indicators, and LCD display provide immediate on-site warnings and system status information. To enhance

decision-making, Python-based machine learning algorithms analyze sensor data patterns and predict possible flood occurrences. By combining embedded systems, IoT connectivity, renewable energy, and AI- driven analytics, the proposed framework provides a reliable, energy-efficient, and scalable solution for proactive flood prediction and early warning, contributing to improved disaster preparedness and public safety.

CHAPTER 2

LITERATURE SURVEY

[1] Title: "AI-Driven Early Warning System for Predicting and Mitigating Urban Flooding: A Comparative Analysis of Real-Time Data and Predictive Algorithms.

Authors: M. Sathiyamoorthy and P. Subramanian

Their study addresses a major real-world problem: urban flooding, which occurs due to rapid urbanization, poor drainage systems, and extreme weather conditions. Traditional flood monitoring systems often rely on static models and delayed data, making them less effective in providing timely alerts. To overcome these limitations, the authors propose an intelligent system that combines real-time environmental data (such as rainfall, water levels, and runoff conditions) with artificial intelligence-based predictive algorithms. A key contribution of their work is the comparative analysis of predictive algorithms. They evaluate different machine learning and AI techniques to determine which models provide higher accuracy and faster predictions.

[2] Title: Fast urban flood modeling informing response decisions: Model development and future perspectives Authors: T. Duan, P. Li, P. Huang.

This literature survey explores the current state of urban flood modeling, highlighting its importance in informing response decisions and mitigating the impacts of flooding in urban areas. The authors review existing models, discussing their strengths and limitations, and identify future research directions to improve flood modeling and response. Key aspects covered include: Advances in data-driven and physics-based modeling approaches Integration of remote sensing and GIS technologies Challenges in model calibration, validation, and uncertainty quantification Emerging trends in real-time forecasting and decision support systems Future perspectives on improving model accuracy, scalability, and usability for emergency response The survey aims to provide researchers, practitioners, and policymakers with an overview of the latest developments and challenges in urban flood modeling, guiding future research and innovation .

[3] Title: Flood resilience through hybrid deep learning: Advanced forecasting for Taipei's urban drainage system Authors: S.M and S.P

This research develops a hybrid deep learning approach to enhance flood forecasting for Taipei's urban drainage system. By combining convolutional neural networks (CNNs) and long short-term memory (LSTM) networks, the model predicts water levels and flood risks in real-time. Key aspects include: The hybrid model improves forecasting accuracy, supporting flood resilience and informed decision-making for Taipei's urban flood management.

[4] Title: Integrating Intelligent Hydro-informatics into an effective Early Warning System for risk informed urban. Authors: L.-C. Chang, M.-T. Yang, and F.-J.

This initiative focuses on integrating intelligent hydro-informatics into an Early warning system for urban flood management. By combining advanced data analytics, machine learning, and real-time hydro-meteorological data, the system risk-informed decision-making for flood mitigation and response. Key aspects include: Hydro-informatics integration: Merges hydrological modeling, data science, and AI for accurate flood forecasting. Early warning system: Provides timely alerts and risk assessments to support proactive flood management. Risk-informed approach: Helps authorities prioritize interventions, evacuation plans, and resource allocation.

[5] Title: Integrating Intelligent Hydro-informatics into an effective Early Warning System for riskinformed urban flood management. Authors: T.Q.Dang.

The Multi-Sensor IoT System for Real-Time Environmental Monitoring focuses on continuous observation of environmental conditions using multiple sensors integrated with Internet of Things technology. The main objective of this system is to collect, transmit, and analyze real-time environmental data to support applications such as weather monitoring, pollution control, agriculture, and disaster management. The proposed system uses a microcontroller-based architecture combined with wireless communication modules to gather environmental information from multiple sensor nodes. Each sensor node continuously collects environmental data and sends it to a central server or cloud platform through IoT communication protocols. This enables remote monitoring and real-time data visualization, allowing users to track environmental conditions from anywhere using mobile or web applications. One of the main advantages of a multi-sensor IoT monitoring system is its ability to provide comprehensive environmental analysis. Instead of relying on a single sensor, the system combines data from several sensors to improve accuracy and reliability. This multi-parameter sensing approach allows the system to detect abnormal environmental changes quickly and generate alerts when parameters exceed predefined thresholds. Such systems are widely used in smart cities, agriculture, disaster monitoring, and environmental protection applications.

CHAPTER 3

SYSTEM ANALYSIS

3.1 PROBLEM STATEMENT

Floods cause severe damage to life and property, destroying homes, infrastructure, and agricultural land. In remote and rural areas, proper monitoring systems are often unavailable, making it difficult to detect rising water levels in time. Traditional methods of flood detection usually provide warnings too late, and manual observation is unreliable, especially during extreme weather conditions. Additionally, power supply issues during floods can disrupt monitoring and warning systems. Currently, there is no intelligent system widely implemented to predict floods in advance and provide early warnings. Most conventional flood detection methods only provide reactive alerts after water levels have already reached dangerous levels, leaving little time for evacuation or preventive measures. Additionally, many systems monitor only a single environmental parameter, such as water level, which does not provide sufficient information for accurate flood prediction. Therefore, there is a need for a smart, automated, and energy-efficient flood monitoring system that can continuously collect environmental data from multiple sensors, transmit the data over long distances, and analyze it using artificial intelligence to provide early warnings before a flood occurs. Integrating technologies such as LoRaWAN for long-range communication, solar power for sustainable operation, and AI-based predictive analytics can significantly improve the reliability and effectiveness of flood prediction systems.

3.2 PROBLEM DESCRIPTION

Floods are among the most destructive natural disasters, causing extensive loss of human life, damage to property, destruction of infrastructure, and severe impact on agricultural land. Every year, flood events disrupt livelihoods and impose significant economic and social burdens, particularly in developing and rural regions. The increasing frequency and intensity of floods due to climate change have further intensified the need for effective flood monitoring and early warning mechanisms. In many remote and rural areas, the absence of reliable flood monitoring infrastructure makes it difficult to detect rising water levels at an early stage. Existing flood detection methods often depend on manual observation or basic monitoring systems, which are prone to human error and become ineffective during extreme weather conditions. These traditional approaches generally provide warnings only after water levels have already reached dangerous limits, leaving little time for preventive action.

or evacuation. Another major challenge is the reliability of power supply during flood situations.

Floods frequently damage electrical infrastructure, causing power outages that disrupt conventional monitoring and communication systems. As a result, warning systems fail at the most critical moments when timely alerts are essential to prevent loss of life and property. Furthermore, most currently deployed systems lack intelligence and predictive capability. To address these challenges, the proposed system introduces an AI-driven LoRaWAN-enabled solar-powered multi-sensor framework. The system integrates multiple environmental sensors, including rain sensors, ultrasonic water-level sensors, pressure sensors, flow sensors, and temperature-humidity sensors, to collect real-time environmental data. The data is transmitted using LoRaWAN technology, which enables long-range, low-power communication suitable for remote areas. The system uses solar energy to power sensor nodes, ensuring continuous operation even in locations without stable electricity.

AI algorithms analyze the collected data to identify patterns and predict potential flood conditions, allowing authorities and communities to receive early warnings through alert systems such as GSM notifications, alarms, or IoT dashboards. By combining IoT, AI, renewable energy, and long-range communication technologies, the proposed framework aims to provide a reliable, scalable, and proactive flood monitoring solution that improves disaster preparedness and reduces the impact of flood events. Furthermore, most currently deployed systems lack intelligence and predictive capability. They are primarily reactive in nature and rely on fixed threshold values rather than analyzing trends or patterns in environmental data. There is no widely implemented system that integrates real-time monitoring with intelligent flood prediction to provide early warnings in advance. These challenges highlight the urgent need for a reliable, autonomous, and intelligent flood monitoring framework capable of operating in remote locations, with minimal power dependency, and providing timely and accurate early warnings to reduce the impact of floods.

3.3 EXISTING SYSTEM

In existing flood monitoring systems, traditional approaches rely on manual observation or basic automated sensors that provide limited data, often with delayed alerts. Many systems use only rain gauges or water-level sensors, without integrating environmental parameters like pressure, flow, temperature, or humidity. Communication is typically local, and there is little predictive capability. Alerts are mostly reactive, and there is no

use of AI for forecasting floods. These limitations result in slower response times, reduced accuracy, and increased risk to communities in flood-prone areas.

3.3.1 Traditional Flood Monitoring Systems

Traditional flood monitoring systems mainly depend on manual observation and hydrological monitoring stations. In these systems, water levels are measured using manual gauges installed near rivers, dams, and reservoirs. The collected data is then reported to authorities for analysis.

- However, these systems have several limitations:

- Data collection is slow and not real-time.
- Monitoring stations are limited in number, which reduces coverage. Flood warnings may be issued too late for effective evacuation.
- Because of these limitations, traditional systems are gradually being replaced by automated monitoring technologies.

3.3.2 IoT-Based Flood Monitoring Systems

Many researchers have proposed IoT-based flood monitoring systems to provide real-time data collection and monitoring. These systems use microcontrollers and sensors such as water level sensors, rain sensors, temperature sensors, and humidity sensors to detect environmental conditions.

For example, an IoT flood monitoring system uses Arduino with sensors like DHT11, rain sensors, and water level sensors to monitor environmental conditions and send alerts through GSM modules and cloud platforms when thresholds are exceeded.

3.3.3 Wireless Sensor Network (WSN) Flood Monitoring Systems

Wireless Sensor Network (WSN) technology is widely used in flood monitoring systems. In these systems, multiple sensor nodes are deployed across flood-prone areas to collect environmental data.

Typical sensors used include:

- Ultrasonic sensors for water level measurement
- Flow sensors for water velocity
- Rain gauges for rainfall detection
- Weather sensors for temperature and humidity

The sensor nodes transmit data to a central gateway or monitoring station where it is analyzed and visualized in dashboards.

3.3.4 Cloud-Based Flood Monitoring Systems

Modern flood monitoring solutions use cloud computing platforms to store and analyze environmental data collected from sensors. These systems allow authorities to monitor flood conditions remotely through web dashboards and mobile applications.

Cloud-based systems provide features such as:

- Historical data storage and analysis
- Real-time visualization of rainfall and water levels
- Automatic alarm notifications

Some flood monitoring platforms also include video monitoring and weather observation systems to track flood conditions more accurately.

3.3.5 AI-Based Flood Prediction Systems

Advanced flood monitoring systems use machine learning and artificial intelligence to analyze environmental data and predict flood events.

These systems use:

- Hydrological data
- Rainfall intensity
- River water levels
- Environmental conditions

AI models can analyze large datasets and predict flood events with high accuracy. Hybrid deep learning models trained with historical flood data can achieve prediction accuracy of nearly 97% in some cases, helping authorities prepare for disasters in advance.

3.4 DISADVANTAGES OF EXISTING SYSTEM

- Short-range and unreliable communication
- GSM, Wi-Fi, and wired networks fail in remote areas
- High power consumption and battery dependency
- Limited sensor coverage
- Poor performance during extreme weather
- No long-range, low-power communication (LoRaWAN)
- Low scalability for large flood-prone areas

3.5 PROPOSED SYSTEM

AI-driven flood prediction system. Uses LoRa WAN for long-range communication. Solar-powered for continuous operation. Multiple sensors monitor

flood parameters. Arduino UNO collects sensor data. Data sent to cloud using NodeMCU.AI (Python) predicts flood risk. Alerts via buzzer, LED, LCD, GSM, and IoT.

3.5.1 Multi-Sensor Environmental Monitoring

The proposed system uses multiple environmental sensors to collect data related to flood conditions.

These sensors monitor parameters such as:

- Rainfall intensity using rain sensors
- Water level using ultrasonic sensors
- Water flow rate using flow sensors
- Atmospheric pressure using pressure sensors
- Temperature and humidity using DHT11 sensors

These sensors continuously capture hydrological and environmental parameters that are important indicators of flood formation. The collected data is sent to the embedded controller for further processing.

3.5.2 Embedded Processing Unit

The sensor data is processed using an embedded microcontroller such as Arduino or NodeMCU.

The controller performs several functions including:

- Sensor data acquisition.
- Data preprocessing and filtering.
- Sensor calibration.
- Packaging data for transmission.

The microcontroller ensures that all sensor readings are processed efficiently before being transmitted to the communication network.

3.5.3 LoRaWAN Communication Network

The proposed system uses LoRaWAN (Long Range Wide Area Network) for wireless data communication between sensor nodes and the central gateway.

Advantages of LoRaWAN include:

- Long communication range (up to several kilometers).
- Low power consumption.
- Reliable transmission in remote areas.
- Ability to support multiple sensor nodes.

The LoRa module (such as SX1278) transmits the collected environmental data from the sensor node to a LoRa gateway, which then forwards the data to the cloud server for analysis.

3.5.4 Solar Power Supply System

Since flood monitoring systems are often deployed in remote or rural areas, the proposed system uses solar panels with rechargeable batteries to power the sensor nodes.

Benefits of solar power include:

- Continuous operation during power outages
- Reduced dependence on electrical grids
- Environmentally sustainable energy source

The solar energy system ensures that the monitoring system remains operational even during extreme weather conditions.

3.5.5 Cloud Data Processing and Storage

The collected data from sensor nodes is transmitted to a cloud server or IoT platform where it is stored and processed.

Cloud platforms provide features such as:

- Real-time data visualization dashboards
- Historical data storage
- Remote monitoring via web or mobile applications

Authorities and disaster management teams can access the system remotely to observe flood conditions.

3.5.6 Artificial Intelligence-Based Flood Prediction

Artificial Intelligence algorithms analyze the collected environmental data to detect patterns that may indicate upcoming flood events.

AI models may include:

- Decision Trees
- Random Forest
- Neural Networks
- Regression models

These models analyze sensor data such as rainfall intensity, water level rise, and flow rate to predict potential flooding before it occurs, enabling proactive disaster management. AI-based systems significantly improve flood prediction accuracy compared to traditional monitoring systems

3.6 ADVANTAGES OF PROPOSED SYSTEM

- Long-range communication using LoRa WAN
- Solar-powered with battery backup
- Multi-sensor based flood monitoring
- AI-based early flood prediction
- Real-time cloud monitoring
- Multiple alert mechanisms
- Reliable during harsh weather

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

AI Driven Lora WAN Enabled Solar Powered Multi Sensor Framework for Proactive Flood Prediction and Early Warning typically incorporate various hardware components to enhance temperature, rain prediction, blow of air . Here's a list of common hardware requirements used in smart vehicle black box using Iot:

- Arduino
- Rain drop sensor
- Ultrasonic sensor
- Pressure sensor
- Nodemcu
- Lora SX1278
- Gps
- Gsm
- Buzzer
- Power supply
- Solar panel 9v
- Dht11 sensor
- Flow sensor

4.2 SOFTWARE REQUIREMENTS

The software for the AI Driven Lora WAN Enabled Solar Powered Multi Sensor Framework for Proactive Flood Prediction and Early Warning plays a crucial role in managing data, processing sensor inputs, providing user interfaces, and enabling communication functionalities.

- Platform : Arduino IDE
- Language : Embedded C

4.3 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal. Three key considerations involved in the feasibility analysis are:

- Financial Feasibility
- Technical Feasibility
- Operational Feasibility

- Environmental Feasibility
- Social Feasibility

4.3.1 Financial Feasibility

The system is built using low-cost embedded components and open-source software platforms, which significantly reduces development and deployment costs. Solar power eliminates recurring electricity expenses, and LoRaWAN reduces communication costs compared to cellular-based solutions. Since the system helps prevent flood damage and loss of life, the long-term benefits outweigh the initial investment. Therefore, the proposed system is economically feasible and suitable for large-scale deployment.

4.3.2 Technical Feasibility

Technical feasibility evaluates whether the required technology and technical resources are available to develop and deploy the system. The proposed system uses widely available and proven technologies such as Arduino UNO, NodeMCU, LoRa SX1278 modules, environmental sensors, GSM, and GPS modules. These components are well-supported, low-cost, and easy to integrate. The use of LoRaWAN enables long-range, low-power communication, making the system suitable for remote and flood-prone regions. Python-based machine learning algorithms are technically feasible due to the availability of open-source libraries and tools. Solar power integration ensures continuous operation, even during power outages. Hence, the project is technically feasible and can be implemented using existing hardware and software technologies.

4.3.3 Operational Feasibility

Operational feasibility assesses whether the system can be effectively used and managed in real-world environments. The system is designed to operate autonomously with minimal human intervention. Real-time monitoring, automated alerts through GSM, and on-site indicators such as buzzer and LCD ensure ease of operation. Authorities and users can monitor data remotely through IoT platforms without requiring specialized technical skills. The solar-powered design ensures reliable operation during emergencies, making the system operationally feasible in flood-prone and remote areas.

4.3.4 Environmental Feasibility

Environmental feasibility examines whether the system supports sustainable and eco-friendly operations. Solar-powered design reduces dependency on conventional power sources. Low-power LoRa communication minimizes energy

consumption. Early flood detection helps protect ecosystems and reduces environmental damage. Conclusion: The proposed framework supports environmentally sustainable disaster monitoring.

4.3.5 Social Feasibility

Provides early flood warnings to communities . Helps government agencies respond faster to disasters. Improves public safety and disaster preparedness.

CHAPTER 5

SYSTEM DEVELOPMENT

5.1 SYSTEM DESIGN

5.1.1 System Architecture

The sensor layer is responsible for continuously monitoring environmental and hydrological parameters related to flood conditions. Multiple sensors are deployed at flood-prone locations to capture diverse data, improving reliability and prediction accuracy.

5.1.1.1 Sensing Layer (Environmental Data Collection)

The sensing layer is responsible for collecting real-time environmental data that indicates potential flood conditions. Multiple sensors are deployed near rivers, drainage systems, and flood-prone areas.

Sensors used in the system include:

- Rain Sensor – Detects rainfall intensity.
- Ultrasonic Sensor – Measures water level in rivers or reservoirs.
- Pressure Sensor (BMP180) – Measures atmospheric pressure changes.
- Flow Sensor – Measures the rate of water flow.
- DHT11 Sensor – Measures temperature and humidity.

These sensors continuously monitor environmental parameters and send the collected data to the microcontroller for processing. Example hardware used in this layer includes DHT11 Temperature and Humidity Sensor and HC-SR04 Ultrasonic Sensor.

5.1.1.2 Embedded Processing Layer (Data Acquisition and Control)

The embedded processing layer collects sensor data and performs initial filtering and processing. The main controller used is Arduino Uno, which acts as the central unit of the monitoring node. It performs the following tasks: Collects real-time data from all sensors. Performs basic preprocessing of data. Determines abnormal environmental patterns. Sends the data to the communication module. A secondary controller such as NodeMCU ESP8266 can be used for IoT connectivity and cloud integration.

5.1.1.3 Communication Layer (LoRaWAN Network)

The communication layer is responsible for transmitting sensor data from remote monitoring stations to a central gateway.

The system uses LoRaWAN technology because it provides:

- Long-range communication (up to 10–15 km)
- Low power consumption
- Reliable data transmission in rural areas

The embedded node uses SX1278 LoRa Module to transmit collected data to the LoRa gateway. The gateway then forwards the data to the internet server.

5.1.1.4 Cloud and AI Processing Layer

After the data reaches the gateway, it is transmitted to the cloud server where advanced analysis takes place. AI algorithms analyze historical and real-time sensor data to detect flood patterns. The analysis includes: Rainfall trend analysis, Rapid water level rise detection, Environmental condition prediction, Flood probability estimation. Machine learning models implemented using Python process the data and generate predictions about potential flood events. Cloud platforms store large datasets and support continuous monitoring and model training.

5.1.1.5 Alert and Notification Layer

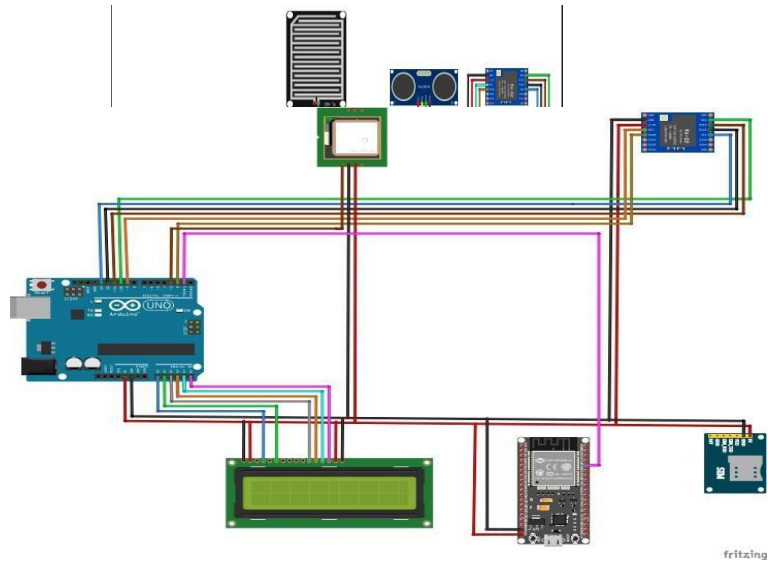
When the AI model detects a possible flood risk, the system generates early warning alerts.

Alerts are sent through:

- SIM800L GSM Module for SMS notifications
- Buzzer alarms for local warnings
- LCD display for real-time monitoring
- IoT dashboards for authorities

These alerts help government agencies and local residents take preventive measures before a flood occurs. Overall Working of the System Sensors collect environmental data. The embedded controller processes sensor data. LoRa modules transmit data to the gateway. Cloud servers analyze data using AI models. If flood risk is detected, alerts are generated and sent to users. These sensors generate real-time analog and digital signals that represent environmental conditions. The use of multi-sensor data fusion allows the system to analyze complex flood patterns rather than relying on a single parameter.

TRANSMITTER:



RECIEVER:

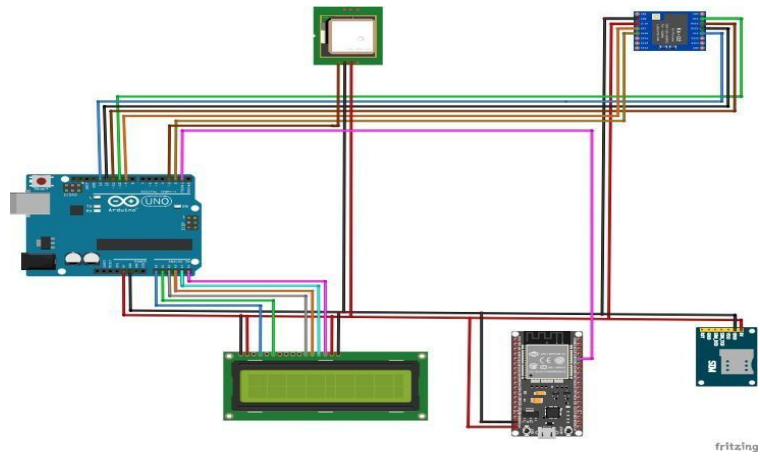


Figure 5.1 System Architecture

BLOCK DIAGRAM:

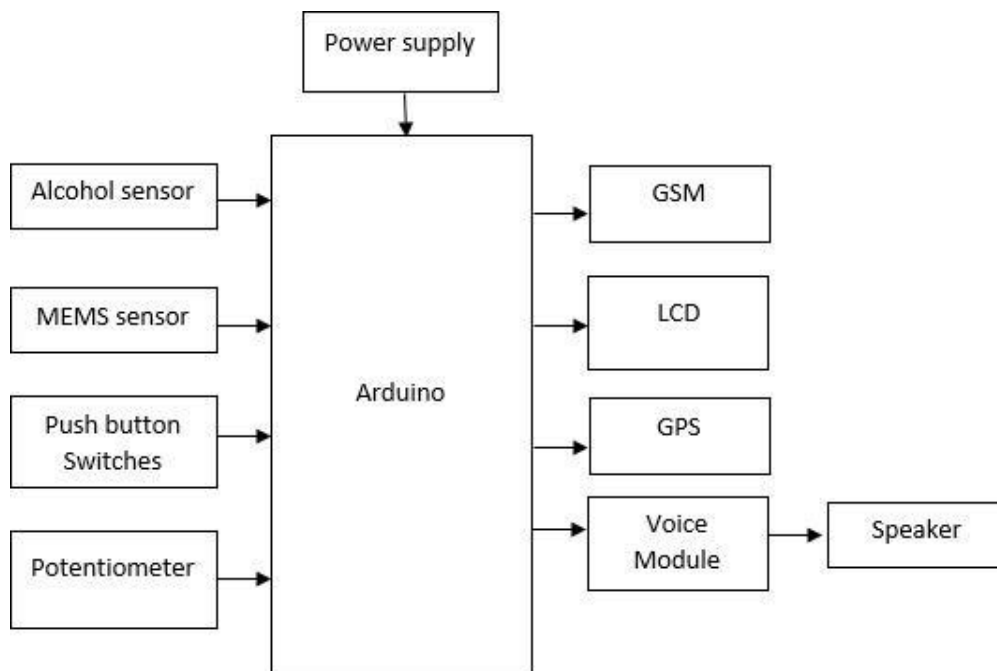


Figure 5.2 Block Diagram

5.1.2 Modules

Our project contains six modules,

1. Arduino Uno
2. Node MCU
3. Sensors
4. Gsm, gps
5. Buzzer
6. Lora sx1278

5.1.2.1 Arduino Uno

Using this module, a versatile microcontroller board, the Arduino serves as the core of the Vehicle Black Box System, processing data from various sensors and modules. It enables seamless integration of components like the Alcohol Sensor, MEMS Sensor, GPS, GSM, and LCD for real-time monitoring. Developed by Arduino LLC, it offers efficient data handling and serial communication support. The Arduino ensures quick accident detection, alert generation, and vehicle status tracking, enhancing road safety.

5.1.2.3 SENSORS

Raindrop sensor



Figure 5.5 Rain drop sensor

A raindrop sensor detects the presence and intensity of rainfall, often used in environmental monitoring systems. It typically consists of a conductive plate that senses raindrops by measuring changes in resistance when water contacts the sensor's surface. As rainfall increases, conductivity rises, allowing the sensor to quantify rainfall intensity. The raindrop sensor is easy to integrate with microcontrollers like Arduino, making it useful in applications such as landslide monitoring, weather stations, and automated irrigation systems for precise rain detection and response. A Raindrop Sensor is an electronic module used to detect rainfall or water droplets. It is widely used in weather monitoring systems, flood detection systems, smart irrigation, and IoT-based environmental monitoring. In an AI-driven LoRaWAN flood detection system, the raindrop sensor helps detect rain intensity and rainfall occurrence, which is an important parameter for predicting possible flood conditions.

Ultrasonic sensor

An ultrasonic sensor measures distance by emitting high-frequency sound waves and detecting their reflection from objects. It calculates distance based on the time taken for the sound waves to return. Widely used in robotics, automotive systems, industrial automation, and home security, ultrasonic sensors are valued for their accuracy, reliability, and ability to detect objects regardless of color or lighting conditions. They are essential for obstacle avoidance, level measurement, and presence detection, making them a versatile choice for various applications. The Ultrasonic Sensor is a device used to measure distance using ultrasonic sound waves. It is commonly used in water level monitoring, obstacle detection, robotics, and flood monitoring systems. In an AI-driven LoRaWAN flood detection system, the ultrasonic sensor measures the

water level in rivers, drains, or reservoirs. By continuously monitoring the rising water level, the system can predict possible flooding conditions and send early warnings.



Figure 5.6 Ultrasonic sensor

Pressure sensor

The BMP180 is a high-precision digital sensor that measures atmospheric pressure, temperature, and altitude.

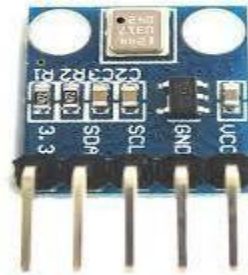


Fig 5.7 Pressure sensor

It's known for its low power consumption, small size, and high resolution, making it ideal for mobile and embedded applications. Widely used in weather monitoring and GPS devices, the BMP180 provides accurate pressure readings and altitude measurements, and easily integrates with microcontrollers like Arduino and Raspberry Pi. The BMP180 sensor works based on the piezoresistive sensing principle.

- Inside the sensor, there is a pressure-sensitive diaphragm.
- When atmospheric pressure changes, the diaphragm deforms slightly.
- This deformation changes the electrical resistance of the sensor.
- The internal circuit converts this change into digital pressure data.

The sensor communicates with the microcontroller using the I²C communication protocol.

Dht11b sensor

The DHT11 sensor is a widely used digital sensor that measures temperature and humidity. It provides reliable and accurate readings by using a thermistor for temperature and a capacitive



Fig 5.8 Dht11 sensor

The DHT11 communicates with microcontrollers through a single-wire digital interface, making it easy to integrate into various projects. It is often used in applications such as weather stations, HVAC systems, and environmental monitoring. With a measurement range of 0-50°C for temperature and 20-80% for humidity, the DHT11 offers a practical solution for basic climate monitoring needs. Its affordability and simplicity make it a popular choice for both hobbyists and professionals. The DHT11 sensor works using two sensing elements:

1. Humidity Sensor (Capacitive Type)

- The sensor contains a moisture-sensitive capacitor.
- When humidity changes, the dielectric constant of the material changes.
- This causes a change in capacitance, which is converted into an electrical signal.

2. Temperature Sensor (Thermistor)

- The temperature is measured using an NTC thermistor.
- The resistance of the thermistor changes with temperature.
- The internal circuit converts the resistance change into digital temperature data.

The sensor has an internal microcontroller that processes the signals and sends digital output data to the microcontroller.

Flow sensor

A flow sensor (more commonly referred to as a “[flow meter](#)”) is an electronic device that measures or regulates the flow rate of liquids and gasses within pipes and tubes. Flow sensors are generally connected to gauges to render their measurements, but they can also be connected to computers and digital interfaces. They are commonly used in HVAC systems, medical devices, chemical factories, and septic systems. Flow

sensors are able to detect leaks, blockages, pipe bursts, and changes in liquid concentration due to contamination or pollution.

1. Basic Concept of Flow Measurement

Flow sensors measure how much fluid passes through a pipe per unit time. The flow rate is usually expressed as:

- Liters per minute (L/min) for liquids
 - Cubic meters per hour (m³/h) for gases
- The general formula used for flow rate is:

$Q = A \times v$ Where:

Q = Flow rate

A = Cross-sectional area of the pipe v = Velocity of the fluid

This means the flow rate depends on pipe size and speed of the fluid.

2. Types of Flow Sensors Used in Pipes and Tubes

1. Turbine Flow Sensor

Example: YF-S201 Water Flow Sensor Working Principle

- A small turbine wheel rotates when liquid flows through the pipe.
- The rotation speed is proportional to the flow rate.
- A Hall effect sensor detects the rotation and generates electrical pulses.

Used for

- Water flow measurement
- Liquid monitoring in pipes
- Embedded systems and IoT projects

3. Electromagnetic Flow Sensor

Working Principle

- Based on Faraday's law of electromagnetic induction.
- When conductive liquid flows through a magnetic field, a voltage is generated.
- The voltage is proportional to the flow velocity.

Advantages

- High accuracy
- No moving parts
- Suitable for dirty liquids
- Applications

- Water treatment plants
- Chemical industries

3. Ultrasonic Flow Sensor Working Principle

- Uses ultrasonic sound waves to measure fluid velocity.
- Sensors are placed on the outside of the pipe.
- The time difference of sound waves determines flow speed.



Fig 5.9 flow sensor

Global System for Mobile Communication(GSM)

Using this module, a wireless communication device, the GSM module enables the Vehicle Black Box System to send real-time alerts via SMS in case of accidents or emergencies. Integrated with Arduino, it transmits location details from the GPS module to emergency contacts for a quick response. It ensures instant communication with authorities or family members, enhancing road safety and accident management. This module plays a vital role in remote monitoring and emergency alert systems. Global System for Mobile



Figure 5.10GSM

Communications (GSM) is a digital cellular communication standard used for transmitting voice, SMS, and data over mobile networks. It is one of the most widely

used mobile communication technologies in the world. In embedded and IoT projects, GSM technology allows microcontroller-based systems to send alerts, SMS notifications, or make calls when certain conditions are detected. In an AI-driven LoRaWAN flood detection system, GSM is used to send early warning messages to authorities and users when flood risk is detected.

Global Positioning System (GPS)

Using this module, a satellite-based navigation system, the GPS module tracks the real-time location of the vehicle. Integrated with Arduino, it continuously provides latitude and longitude coordinates, which can be sent via GSM in case of accidents. This enables quick emergency response by sharing the vehicle's precise location with authorities or family members. The GPS module plays a crucial role in vehicle tracking, accident detection, and navigation, enhancing road safety and monitoring. The Global Positioning System (GPS) is a satellite-based navigation system that provides accurate location, time, and velocity information anywhere on Earth. It uses a network of satellites orbiting the Earth to determine the exact position of a receiver. GPS is widely used in navigation systems, smartphones, vehicle tracking, disaster management, and IoT-based monitoring systems.



Figure 5.11 GPS

Lora sx1278

LoRa (short for long range) is a spread spectrum modulation technique derived from chirp spread spectrum (CSS) technology. Semtech's LoRa devices and wireless radio frequency technology is a long range, low power wireless platform that has become the de facto technology for Internet of Things (IoT) networks worldwide. LoRa devices and the open LoRaWAN® protocol enable smart IoT applications that solve some of the biggest challenges facing our planet: energy management, natural resource reduction, pollution control, infrastructure efficiency, disaster prevention, and more. Semtech's LoRa devices and the LoRaWAN protocol have amassed several

hundred known uses cases for smart cities, smart homes and buildings, smart agriculture, smart metering, smart supply chain and logistics, and more.

With well over 100 million devices connected to networks in 100 countries and growing, LoRa devices are the DNA of IoT, creating a Smarter Planet. The SX1278 LoRa module is a long-range wireless communication module based on LoRa (Long Range) technology. It is designed for low-power, long-distance data transmission, making it ideal for IoT, environmental monitoring, and smart city applications. In an AI-driven LoRaWAN flood detection system, the SX1278 module is used to transmit sensor data from remote monitoring stations to a central server or gateway over long distances. LoRa (Long Range) is a wireless communication technology designed for long-distance communication with very low power consumption. Key characteristics of LoRa:

- Long communication range (2–15 km)
- Low power consumption
- Low data rate

LoRa uses Chirp Spread Spectrum (CSS) modulation, which allows signals to travel long distances while maintaining reliable communication.

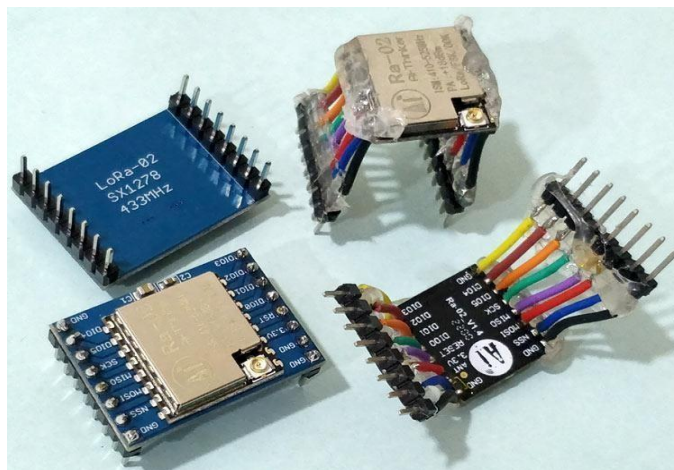


Figure 5.13 Lora sx1278

5.1.2.4 SOLAR POWER

A solar power system is used to provide continuous and renewable energy to the flood monitoring station. In remote flood-prone areas, grid electricity may not be available. Therefore, solar energy is used to power sensors, microcontrollers, and communication modules. In the AI-driven LoRaWAN flood detection and early warning system, the solar power unit supplies energy to devices such as Arduino Uno

R3, NodeMCU ESP8266 Development Board, and the SX1278 LoRa Transceiver Module, enabling continuous environmental monitoring.

1. Working Principle of Solar Power System)

The solar power system converts sunlight into electrical energy using photovoltaic technology. Step-by-Step Process

-Solar Panel

- Captures sunlight and converts it into DC electricity using photovoltaic cells.

-Charge Controller

- Regulates the voltage and current from the solar panel.
- Prevents battery overcharging and deep discharge.

-Battery Storage

- Stores the generated electrical energy.
- Provides power during night or cloudy weather.

-Power Supply to System

- The stored energy powers the sensors, microcontroller, LoRa module, and alert system.

2. Continuous Operation

The system runs continuously for 24/7 flood monitoring.

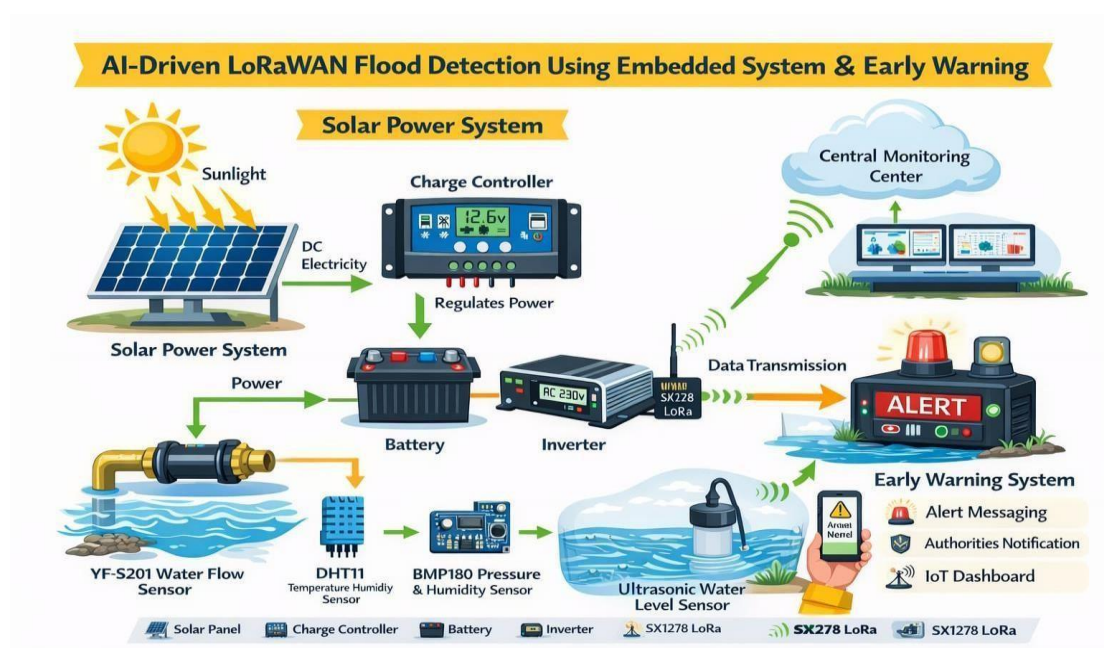


Fig 5.14 solar power

5.1.2.5 Buzzer:

A buzzer is a simple yet effective electromechanical device that produces sound by vibrating a diaphragm or membrane when an electrical current passes through it. A buzzer or beeper is an audio signaling device, which may be mechanical, electromechanical, or piezoelectric. Typical uses of buzzers and beepers include alarm devices, timers and confirmation of user input such as a mouse click or keystroke. Buzzer is an integrated structure of electronic transducers, DC power supply, widely used in computers, printers, copiers, alarms, electronic toys, automotive electronic equipment, telephones, timers and other electronic products for sound devices. Active buzzer 5V Rated power can be directly connected to a continuous sound, this section dedicated sensor expansion module and the board in combination, can complete a simple circuit design, to "plug and play."



Figure 5.15 Buzzer

A buzzer is an electronic audio signaling device that produces sound when electrical power is applied.

It is commonly used in alarm systems, warning devices, and embedded systems to provide audible alerts. In the AI-driven LoRaWAN flood detection and early warning system, the buzzer acts as a local alarm device. When the system detects a dangerous water level or flood condition, the buzzer generates a loud sound to warn people nearby.

The buzzer is usually connected to microcontrollers such as:

- Arduino Uno R3
- NodeMCU ESP8266 Development Board

5.1.3 Algorithm

initializeSystem() while true:

sensorData = readSensorData()

processedData = processSensorData(sensorData)

If checkForAlerts(processedData): generateAlerts(processedData)

sendDataToServer(processedData) Display(processedData)

if isFloodRisk(processedData): raiseBuzzer() sendAlertNotification()

5.1.3 UML Diagrams

5.1.3.1 Introduction to UML

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML. The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non- software systems. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation.

In the future, some form of method or process may also be added to; or associated with, UML. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing object-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

GOALS:

The Primary goals in the design of the UML are as follows:

- 5.1.3.1.1 Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- 5.1.3.1.2 Provide extendibility and specialization mechanisms to extend the core concepts.
- 5.1.3.1.3 Be independent of particular programming languages and development process.
- 5.1.3.1.4 Provide a formal basis for understanding the modeling language.
- 5.1.3.1.5 Encourage the growth of Object Orient tools market.
- 5.1.3.1.6 Support higher level development concepts such as collaborations, frameworks, patterns and components.
- 5.1.3.1.7 UML aims to standardize the language used in software design, ensuring consistent communication across stakeholders.
- 5.1.3.1.8 It provide graphical representations to visually depict the architecture, structure, and behavior of software systems, enhancing understanding.
- 5.1.3.1.9 UML serves as a means to specify and document system requirements, design decisions, and behavior in a structured manner.
- 5.1.3.1.10 Through various modeling techniques, UML allows developers to create models that capture different aspects of a system's functionality and design.

Overall, the goals of UML revolve around improving communication, facilitating design and analysis, promoting standardization, and supporting the development of complex software systems.

5.1.3.2 Class Diagram

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an “is-a” or “has-a” relationship. Each class in the class diagram may be capable of providing certain functionalities. These functionalities provided by the class are termed “methods” of the class. Apart from this, each class may have certain “attributes” that uniquely identify the class. Class diagrams also help in A Class Diagram is a type of UML (Unified Modeling Language) diagram used in software engineering to represent the structure of a system. It shows the classes, attributes, methods (functions), and relationships between different classes in a system. Class diagrams .

understanding the static structure of the system. They play a key role in object-oriented analysis and design by modeling real-world entities. The visibility of attributes and methods (public, private, protected) is clearly represented. A Class Diagram is a type of UML (Unified Modeling Language) diagram used in software engineering to represent the structure of a system. It shows the classes, attributes, methods (functions), and relationships between different classes in a system.

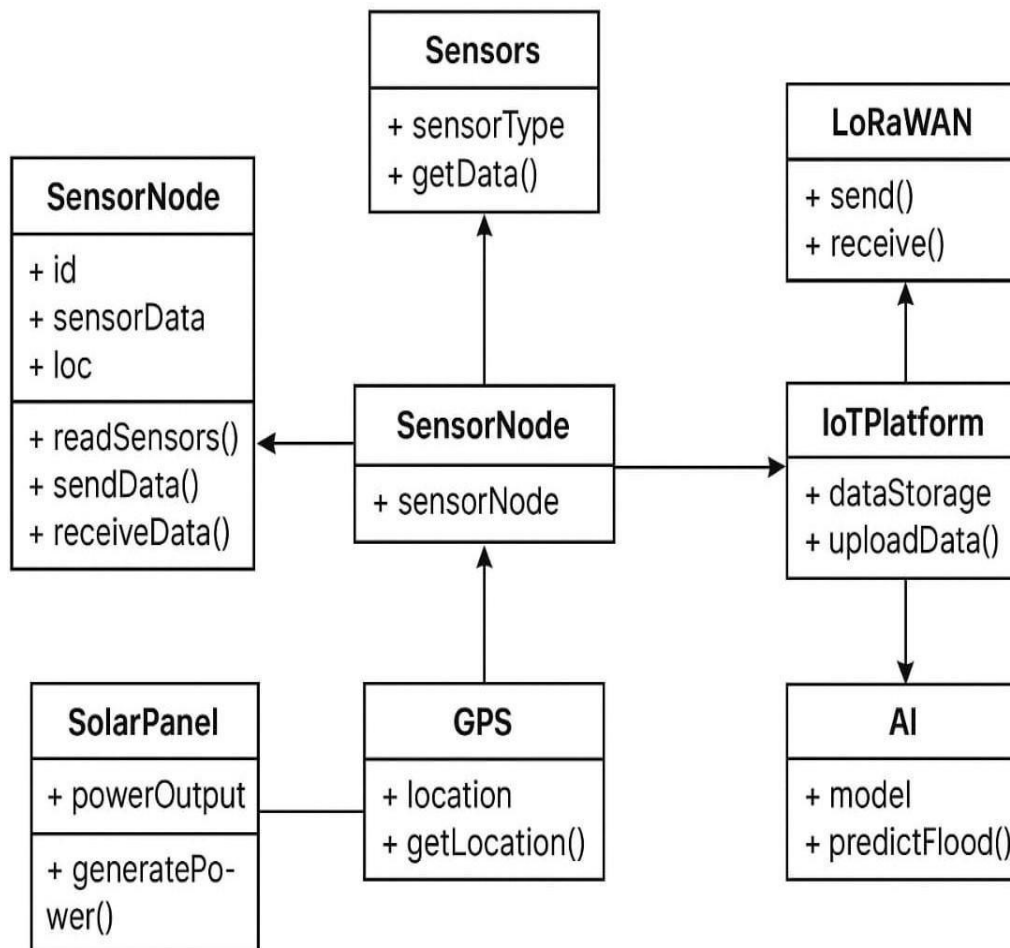


Figure 5.16: Class Diagram

widely used in object-oriented programming (OOP) to design and visualize how different components of a system interact. In an AI-driven LoRaWAN flood detection system, a class diagram can represent the software.

5.1.3.3 Use case Diagram

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted. Use case diagrams help stakeholders understand system interactions at a high level. They serve as a communication tool between developers and non-technical users. These diagrams are essential during the requirement gathering and system design phases. They also assist in identifying system boundaries and validating functionality expectations.

In an AI-driven LoRaWAN Flood Detection and Early Warning System, the use case diagram describes how different users and components interact with the system to monitor environmental conditions, analyze data, and provide early warnings about potential flooding. The diagram represents the system from a functional perspective by identifying the external actors and showing the services or actions the system provides to them. In this project, the system integrates Internet of Things (IoT) sensors, long-range wireless communication through LoRaWAN, cloud data processing, and artificial intelligence algorithms to predict floods before they occur and to warn authorities and the public in advance. The first important actor in the system is the IoT sensor device deployed in flood-prone areas such as riverbanks, drainage channels, dams, and reservoirs. These sensors continuously measure environmental parameters like water level, rainfall intensity, humidity, and sometimes water flow velocity.

Once the measurements are captured, the sensor node processes the readings and transmits them wirelessly. Because flood monitoring areas are often remote and may lack traditional communication infrastructure, the system uses LoRaWAN technology, which allows sensors to send data over long distances while consuming very little power. Through the use case interaction, the sensor device periodically collects environmental data and sends it to the LoRaWAN gateway. The LoRaWAN gateway acts as a communication bridge between the field devices and the central system. When the gateway receives data packets transmitted from the sensors, it forwards them to the network server or cloud platform through an internet connection such as Ethernet, cellular networks, or satellite communication. In the use case diagram, the gateway interacts with the flood monitoring system by transferring sensor data for further

processing. This interaction ensures that real-time environmental information reaches the central processing system where it can be stored and analyzed.

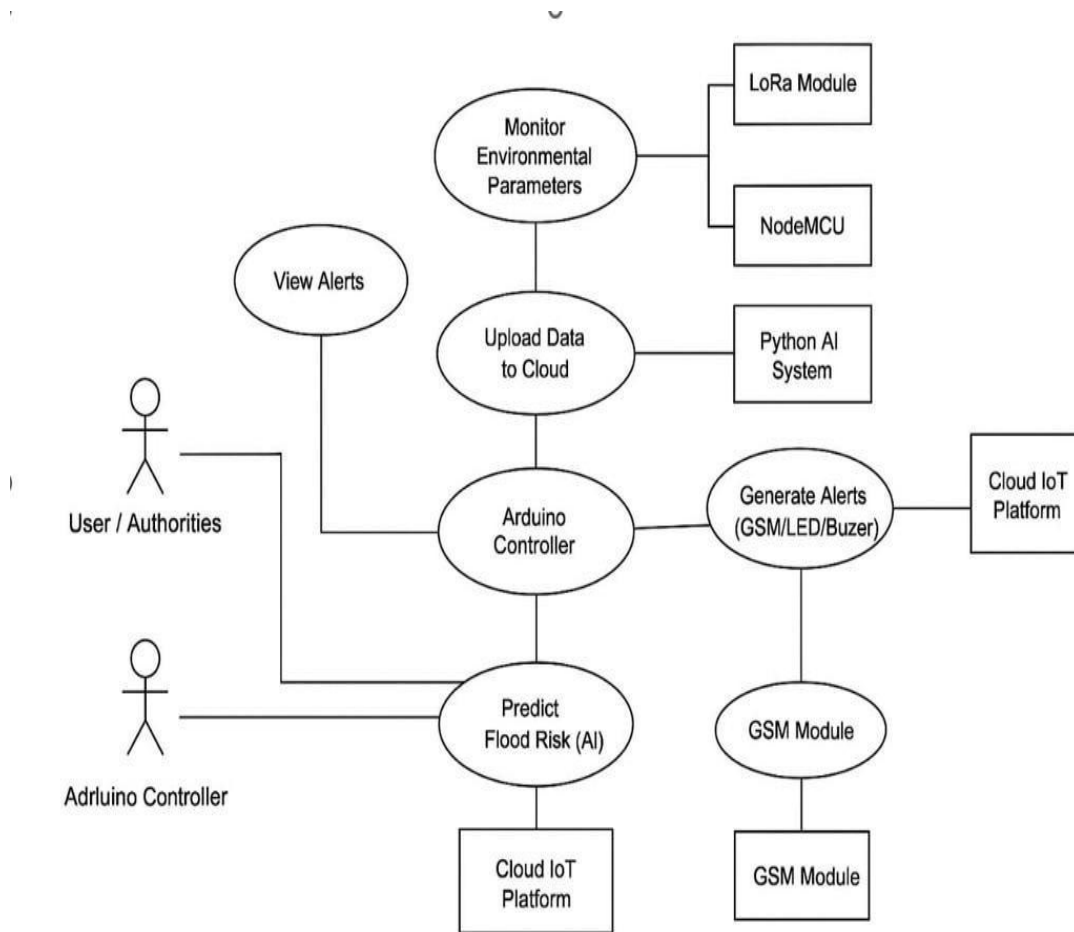


Figure 5.17: Use Case Diagram

5.1.3.4 Sequence Diagram

A sequence diagram represents the interaction between different objects in the system. The important aspect of a sequence diagram is that it is time-ordered. This means that the exact sequence of the interactions between the objects is represented step by step. Different objects in the sequence diagram interact with each other by passing “messages”. These messages can represent method calls, signals, or data transfers. These messages can represent method calls, signals, or data transfers. These messages can represent method calls, signals, or data transfers. Sequence diagrams are essential for visualizing the dynamic behavior of a system. They help in identifying potential design issues early in the development process. In an AI-driven LoRaWAN Flood Detection and Early Warning System, the sequence diagram describes the step-

by-step interaction between different components of the system over time. It shows how messages are exchanged from the moment environmental data is sensed until a flood warning reaches authorities and the public. The diagram mainly focuses on the chronological order of operations and the communication between system elements such as sensors, gateways, servers, artificial intelligence modules, and users.

The sequence diagram helps in understanding the runtime behavior of the system and how different components collaborate to detect flood risks and generate alerts. The sequence begins with the environmental sensing process. IoT sensor nodes installed near rivers, reservoirs, or drainage systems continuously monitor environmental parameters such as water level, rainfall intensity, and sometimes water flow speed. These sensors operate periodically, meaning they wake up at predefined intervals to measure the surrounding environmental conditions. Once the sensor captures the water level value, it temporarily processes the data locally within the microcontroller.

This processing may include filtering noise, converting analog readings into digital values, and formatting the data packet according to the communication protocol used in the system. After the sensor prepares the data packet, the next interaction occurs between the sensor node and the LoRaWAN gateway. Using LoRa communication technology, the sensor transmits the collected data wirelessly over long distances. LoRaWAN is particularly useful in flood monitoring because it allows low-power devices to send data over several kilometers without requiring cellular networks. When the sensor sends the data packet, the LoRaWAN gateway receives the signal and verifies its integrity. If the data packet is successfully received, the gateway acknowledges the transmission and forwards the information to the network server using an internet connection such as Ethernet, Wi-Fi, or cellular networks.

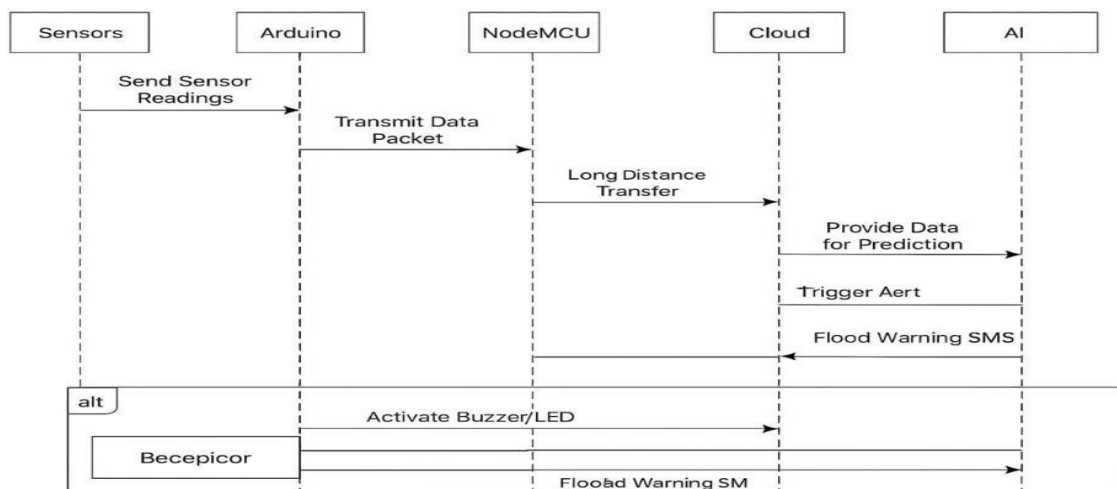


Figure 5.18: Sequence Diagram

5.1.3.5 Activity Diagram

An activity diagram is a type of UML (Unified Modeling Language) diagram used to model the workflow or processes within a system. It illustrates the dynamic flow of control and data between various activities and actions. Similar to state diagrams, activity diagrams help visualize the sequence of activities but are more focused on the business processes and workflows. They are particularly useful for modeling the internal processes of a system, use cases, or even complex business logic. In an AI-driven LoRaWAN Flood Detection and Early Warning System, the activity diagram represents the overall workflow of the system and shows how different operations are performed step by step to monitor environmental conditions and generate flood warnings.

It illustrates the flow of activities starting from environmental sensing and ending with alert notifications to authorities and the public. Unlike sequence diagrams that focus on interactions between components over time, the activity diagram focuses on the logical flow of actions, decisions, and processes within the system. The activity begins with the system initialization stage, where the IoT sensor nodes, LoRaWAN gateway, cloud server, and monitoring platform are activated. During this stage, the sensors are configured to start collecting environmental parameters at specific intervals. These sensors are deployed in flood-prone regions such as riverbanks, reservoirs, drainage channels, and low-lying urban areas. After the system starts operating, the sensors enter a continuous monitoring cycle in which they periodically measure

environmental conditions. The next activity in the workflow involves environmental data collection. Each IoT sensor measures parameters such as water level, rainfall intensity, humidity, and sometimes river flow velocity. These measurements are captured through specialized sensors connected to microcontroller units.

Once the sensor collects the data, the microcontroller processes the readings by converting raw analog signals into digital values. Basic filtering and formatting are also applied to remove noise and prepare the data for wireless transmission. After processing the sensor readings, the system proceeds to the data transmission activity. In this step, the sensor node sends the collected information through LoRa communication technology. LoRaWAN is specifically designed for long-range, low-power communication, which makes it suitable for remote flood monitoring environments where traditional communication infrastructure may not be available. The transmitted data packet travels wirelessly and is received by the nearest LoRaWAN gateway installed within the coverage area. Once the gateway receives the sensor data, the activity flow moves to data forwarding and network communication. The gateway acts as an intermediary device that forwards the received data to the network server through an internet connection. The server verifies the integrity of the received packet and performs authentication to ensure that the data originates from a registered sensor device. After verification, the server forwards the decoded data to the application server or cloud platform. The next stage in the activity diagram involves data storage and management. The application server receives the decoded sensor readings and stores them in a centralized cloud database. The database maintains both current and historical environmental data. This stored information is important because it allows the system to analyze long-term patterns in water levels and rainfall intensity. The data storage activity ensures that every incoming sensor reading is properly recorded and organized for future analysis and monitoring.

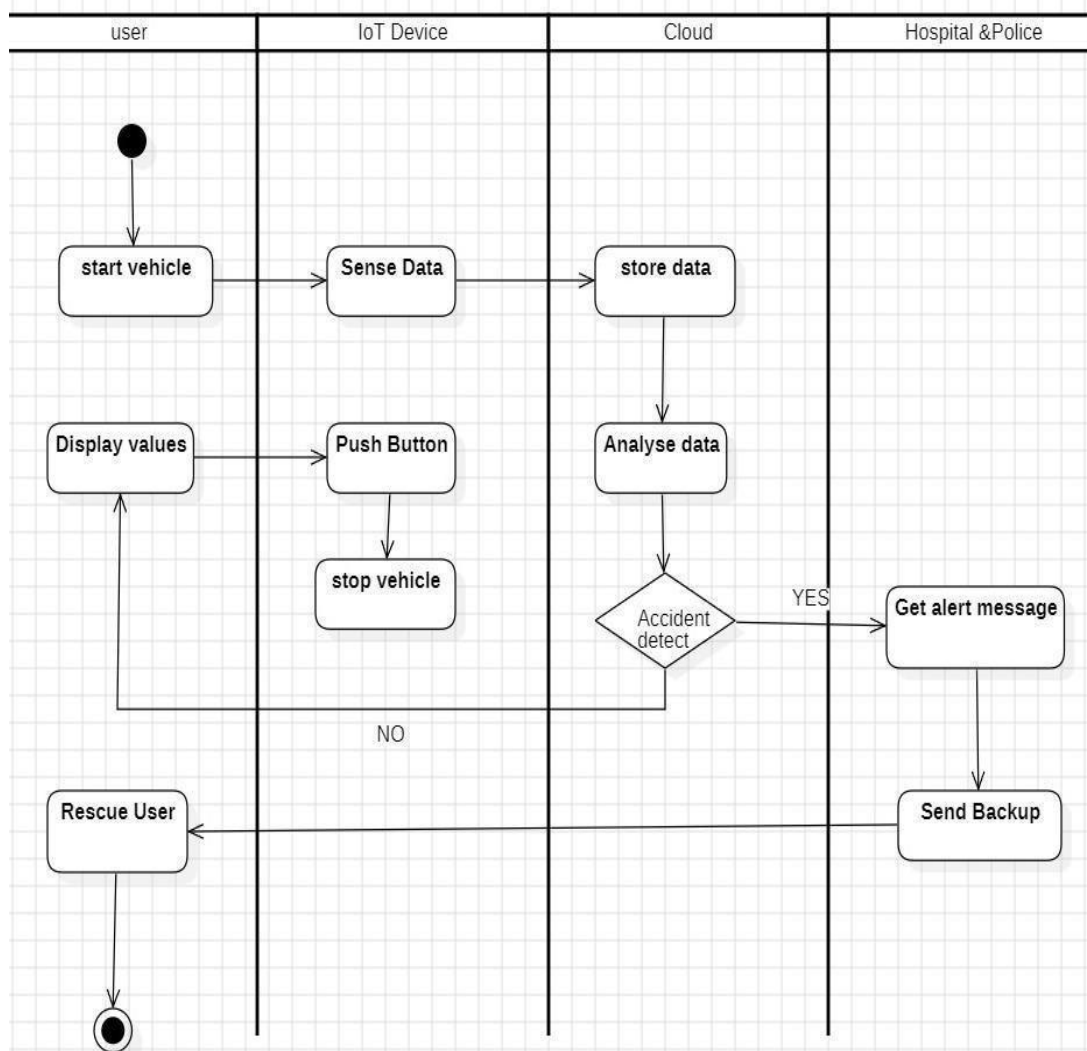


Figure 5.19: Activity Diagram

5.1.4.5 Deployment Diagram

The deployment diagram visualizes the physical hardware on which the software will be deployed. It portrays the static deployment view of a system. It involves the nodes and their relationships. It ascertains how software is deployed on the hardware. It maps the software architecture created in design to the physical system architecture, where the software will be executed as a node. Since it involves many nodes, the relationship is shown by utilizing communication paths. Deployment diagrams are crucial for understanding system performance and scalability. They also help in planning network infrastructure and resource allocation efficiently. In an AI-driven LoRaWAN Flood Detection and Early Warning System, the deployment diagram describes the physical architecture of the system and shows how hardware devices, servers, and software components are distributed across different nodes in the

network. It represents where the system components are deployed and how they communicate with each other to perform flood monitoring, prediction, and alert generation.

The deployment diagram focuses on the real-world infrastructure required for the system, including sensor devices, communication gateways, cloud servers, and user interfaces. The deployment process begins with the sensor node layer, which represents the physical devices installed in flood-prone areas such as riverbanks, drainage channels, dams, reservoirs, and urban flood zones. Each sensor node consists of several hardware components, including water level sensors, rainfall sensors, microcontrollers, and LoRa communication modules. The water level sensor continuously measures the height of water in the monitored location, while the rainfall sensor measures precipitation levels that could contribute to flooding. These sensors are connected to a microcontroller unit that processes the collected environmental data. The microcontroller prepares the data packet and transmits it through the LoRa communication module. Because LoRa technology consumes very little power, these sensor nodes can operate for long periods using batteries or solar power systems. In the deployment diagram, the sensor nodes are shown as distributed devices deployed across multiple geographical locations to provide comprehensive monitoring coverage. The next important node in the deployment structure is the LoRaWAN gateway. This device acts as a bridge between the sensor nodes and the internet-based infrastructure. Multiple sensor nodes transmit their data wirelessly to the gateway using the LoRa communication protocol.

The gateway receives signals from many sensors within its communication range, which can extend several kilometers depending on environmental conditions. After receiving the data packets, the gateway aggregates the information and forwards it to the LoRaWAN network server through an internet connection. This connection may use Ethernet, Wi-Fi, cellular networks, or satellite communication depending on the available infrastructure in the deployment area. In the deployment diagram, the gateway is placed between the sensor layer and the cloud network layer to represent its role as an intermediary communication device. Once the data reaches the internet infrastructure, it is transmitted to the LoRaWAN network server, which is responsible for managing communication between gateways and application services. The network server performs tasks such as device authentication, message decoding, and packet filtering. Each sensor device registered in the system has a unique identifier and

security credentials, and the network server verifies these credentials before accepting the data. After authenticating and decoding the data packets, the network server forwards the processed information to the application server where the main flood monitoring functions are executed. The cloud application server is another important node in the deployment diagram. This server hosts the main software components of the flood detection system. These components include the data processing module, database management system, artificial intelligence prediction engine, and alert management system. When the application server receives environmental data from the network server, it stores the information in a centralized database. The database contains both real-time sensor readings and historical environmental records. Maintaining historical data is necessary for analyzing long-term patterns in water level changes and rainfall trends.

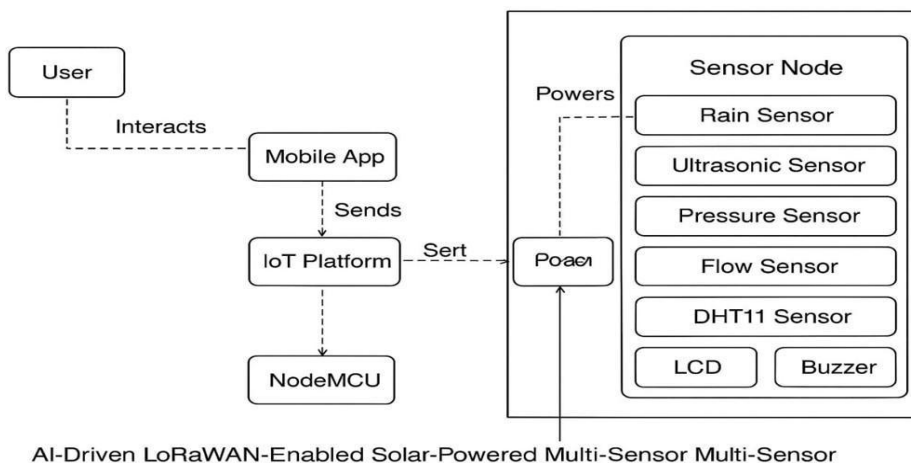


Figure 5.20: Deployment Diagram

5.2 SYSTEM IMPLEMENTATION

5.2.1 Software Description

An embedded system is one kind of a computer system mainly designed to perform several tasks like to access, process, and store and also control the data in various electronics-based systems. Embedded systems are a combination of hardware and software where software is usually known as firmware that is embedded into the hardware. One of its most important characteristics of these systems is, it gives the o/p within the time limits. Embedded systems support to make the work more perfect and convenient. So, we frequently use embedded systems in simple and complex devices too. The applications of embedded systems mainly involve in our real life for several

devices like microwave, calculators, TV remote control, home security and neighbourhood traffic control systems, etc. The embedded system software is written to perform a specific function. It is typically written in a high level format and then compiled down to provide code that can be lodged within a non-volatile memory within the hardware. An embedded system software is designed to keep in view of the three limits:

- Availability of system memory
- Availability of processor's speed
- Need to limit power dissipation

The software component of the AI-driven LoRaWAN Flood Detection and Early Warning System plays a crucial role in collecting, processing, analyzing, and distributing flood-related information. The software is responsible for integrating IoT devices, communication networks, artificial intelligence algorithms, databases, and user interfaces into a unified system that can monitor environmental conditions in real time and generate early flood warnings. It acts as the central control system that manages the flow of data from sensors to decision-making authorities and local communities. The system software begins its operation by receiving environmental data from IoT sensor nodes installed in flood-prone areas. These sensors measure parameters such as water level, rainfall intensity, humidity, and river flow rate. Each sensor node contains a microcontroller and a LoRa communication module that sends collected readings to a nearby LoRaWAN gateway.

On the software side, a LoRaWAN network server manages communication between the gateway and the application server. The network server authenticates devices, decodes data packets, filters duplicates, and forwards valid data to the main application layer. Once the data reaches the application server, it is processed and stored in a centralized cloud database. The database is designed to handle both real-time data streams and large volumes of historical environmental records. Data storage is an important part of the system because historical datasets are used to analyze trends in water levels and rainfall patterns over time. The database system may use structured query-based storage systems or cloud-based data platforms that allow efficient retrieval and analysis of sensor readings. The data processing module in the software performs preprocessing operations on incoming sensor data. These time data streams and large volumes of historical environmental records. Data storage is an important part of the system because historical datasets are used to analyze trends in water levels and rainfall patterns over time. The database system is t

operations include data validation, filtering of noise, normalization, and timestamp synchronization.

Data cleaning ensures that incorrect or missing values do not affect the accuracy of the prediction algorithms. The processed data is then organized into structured datasets that can be used for monitoring and predictive analysis. The most important component of the system software is the Artificial Intelligence prediction module. This module uses machine learning algorithms to analyze environmental data and identify patterns that may indicate the possibility of flooding. The AI system can be trained using historical flood records, rainfall statistics, and river water level data. By comparing real-time sensor readings with historical trends, the AI model can detect abnormal increases in water levels or continuous heavy rainfall conditions. The prediction module calculates the probability of flooding and classifies the situation into risk categories such as low risk, moderate risk, or high risk. This predictive capability allows the system to provide early warnings before the flood actually occurs.

5.2.2 Arduino IDE

An Arduino IDE where IDE stands for Integrated Development Environment – An official software introduced by Arduino.cc, that is mainly used for writing, compiling and uploading the code in the Arduino Device. Almost all Arduino modules are compatible with this software that is an open source and is readily available to install and start compiling the code on the go. Arduino IDE is an open-source software application used for writing, compiling, and uploading programs to Arduino-compatible microcontroller boards. It is designed to provide an easy and user-friendly development environment for programming embedded systems and electronic devices. The software supports programming in a simplified version of the C and C++ programming languages, which allows developers, students, and engineers to control hardware components such as sensors, motors, displays, and communication modules. The Arduino IDE provides an integrated platform where users can write code, verify it for errors, and upload the compiled program to an Arduino board through a USB connection or other supported interfaces. The interface of the software is simple and consists mainly of a text editor for writing code, a message area for displaying compilation messages, a toolbar with quick access buttons, and a serial monitor for viewing communication data between the computer and the microcontroller. One of the main features of the Arduino IDE is its sketch- based programming structure.

Arduino terminology, a program is called a sketch. Every sketch contains two important functions: `setup()` and `loop()`.

The `setup()` function runs once when the program starts and is used to initialize variables, sensors, and communication interfaces. The `loop()` function runs continuously after setup and contains the main instructions that control the behavior of the connected hardware. The Arduino IDE also includes a built-in compiler that converts the written code into machine language that the microcontroller can understand. When the user presses the verify or compile button, the IDE checks the code for syntax errors and compiles it. After successful compilation, the program can be uploaded directly to the connected Arduino board using the upload function. This process transfers the compiled code to the microcontroller memory so that it can run independently without needing a continuous computer connection. Another important feature of the Arduino IDE is the library management system. Libraries are collections of prewritten code that simplify the process of interacting with hardware devices and communication protocols. Users can easily add libraries to support various sensors, communication modules, displays, and other electronic components. These libraries reduce the complexity of programming and allow developers to quickly implement advanced functions without writing code from scratch.

How to install Arduino IDE

You can download the Software from Arduino main website. As I said earlier, the software is available for common operating systems like Linux, Windows, and MAX, so make sure you are downloading the correct software version that is easily compatible with your operating system.

The IDE environment is mainly distributed into three sections:

- Menu Bar
- Text Editor
- Output Panel

When you buy the new Arduino Module, the bootloader is already installed inside the controller. However, if you intend to buy a controller and put in the Arduino module, you need to burn the bootloader again inside the controller by going to the Tools section and selecting the burn bootloader.

5.2.3 Hardware Description

To make software to work with embedded systems we need to bring software and hardware together. For this purpose we need to burn our source code into microprocessor or microcontroller which is a hardware component and which takes care of all operations to be done by embedded system according to our code. An embedded system is one kind of a computer system mainly designed to perform several tasks like to access, process, and store and also control the data in various electronics-based systems. Embedded systems are a combination of hardware and software where software is usually known as firmware that is embedded into the hardware. One of its most important characteristics of these systems is, it gives the o/p within the time limits. Embedded systems support to make the work more perfect and convenient.

So, we frequently use embedded systems in simple and complex devices too. The applications of embedded systems mainly involve in our real life for several devices like microwave, calculators, TV remote control, home security and neighbourhood traffic control systems, etc. Generally we write source codes for embedded systems in assembly language, but the processors run only executable files. The process of converting the source code representation of your embedded software into an executable binary image involves three hardware setup typically involves a combination of hardware and software components working together. The hardware of an AI-driven LoRaWAN Flood Detection and Early Warning System consists of several electronic components that work together to collect environmental data, transmit it over long distances, and support real-time monitoring for flood prediction. These hardware components are mainly divided into sensing devices, processing units, communication modules, power supply systems, and gateway infrastructure.

Each component performs a specific function in the system to ensure accurate monitoring of flood-prone areas. At the core of the hardware setup is the microcontroller board, which acts as the main processing unit of the sensor node. A commonly used board for this purpose is the Arduino Uno or other compatible boards such as Arduino Nano or ESP32. The microcontroller is responsible for reading data from sensors, processing the collected measurements, and controlling communication with the LoRa module. It executes the embedded program that determines how frequently the sensors collect data and how the data is transmitted to the network. The

microcontroller also performs basic processing tasks such as filtering noise and converting raw sensor values into usable measurements. One of the most important sensing components in the system is the water level sensor.

This sensor measures the height of water in rivers, reservoirs, drainage systems, or flood-prone areas. Water level sensors can operate using different technologies such as ultrasonic sensing, float-based measurement, or pressure sensing. When the water level rises due to heavy rainfall or upstream flooding, the sensor detects the change and sends the measurement to the microcontroller. Monitoring water level is essential because rapid increases in water depth often indicate a potential flood situation. Another important sensing device is the rainfall sensor, such as the YL-83 Rain Sensor Module. This sensor measures the intensity or presence of rainfall. Rainfall information is important because heavy or continuous rain is one of the main causes of flooding. By combining rainfall measurements with water level data, the system can better understand environmental conditions and improve the accuracy of flood predictions.

For wireless communication, the system uses a LoRa transceiver module such as the SX1278 LoRa Module. This module enables long-range, low-power communication between sensor nodes and a gateway. LoRa technology allows sensor nodes to transmit data over distances that can reach several kilometers, which makes it suitable for monitoring remote or rural areas where conventional communication networks may not be available. The LoRa module connects to the microcontroller through communication interfaces such as SPI and is responsible for sending sensor readings to the LoRaWAN gateway. The LoRaWAN gateway is another key hardware component in the system. The gateway receives wireless data packets from multiple sensor nodes and forwards them to the internet or cloud server. The gateway acts as a bridge between the local LoRa communication network and the global internet infrastructure. Once the gateway receives the sensor data, it transmits the information to the cloud-based application server using Ethernet, Wi-Fi, or cellular communication networks. This allows environmental data collected in remote locations to be analyzed in centralized servers where AI prediction models operate.

5.2.4 Database Description

NodeMCU is an open-source firmware and development kit based on the ESP8266 WiFi module. It combines the power of the ESP8266 microcontroller with

the convenience of Node.js programming, providing an easy-to-use platform for Internet of Things (IoT) development. NodeMCU supports the Lua scripting language, allowing developers to quickly prototype and deploy IoT applications without the need for extensive low-level programming. With built-in WiFi capabilities, GPIO pins, and support for various sensors and actuators, NodeMCU is widely used for creating smart devices, home automation systems, and IoT projects. The database in an AI-driven LoRaWAN Flood Detection and Early Warning System is a critical component that stores, organizes, and manages all the environmental and operational data generated by the system. The database acts as the central storage location where sensor readings, historical records, prediction results, and system logs are maintained.

This stored information is later used for real-time monitoring, data analysis, machine learning predictions, and alert generation. Without a properly designed database, it would be difficult to analyze patterns or maintain accurate flood prediction models. The main purpose of the database is to store sensor data collected from IoT devices deployed in flood-prone areas. These sensors measure environmental parameters such as water level, rainfall intensity, humidity, temperature, and river flow speed. Each sensor node periodically transmits its readings through the LoRaWAN communication network to the cloud server. When the data reaches the server, it is inserted into the database with important attributes such as sensor ID, timestamp, location, and measured values. Storing data with timestamps allows the system to track how environmental conditions change over time. The database also stores historical environmental records.

Historical data is extremely important in flood prediction systems because artificial intelligence models require past observations to learn patterns associated with flooding events. By analyzing previous water level fluctuations, rainfall patterns, and past flood occurrences, the system can train machine learning algorithms to detect abnormal conditions and predict possible floods. Therefore, the database must be capable of handling large volumes of time-series data collected over long periods. Another important function of the database is storing sensor device information. Each sensor node in the system must be registered and identified within the database. This device information typically includes the sensor ID, installation location, sensor type, calibration details, and operational status. Maintaining this information allows the system to track which devices are active, detect faulty sensors, and manage device maintenance efficiently. The database also maintains prediction

and analysis results generated by the AI module. When the artificial intelligence system analyzes environmental data and predicts flood risk, the prediction results are stored in the database. These records may include the predicted flood probability, risk level classification (such as low, medium, or high), and the time of prediction. Storing these predictions allows authorities to review past warnings and evaluate the performance of the prediction models over time.

5.2.5 Sample Code

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h> #include <DHT.h>
#include <Adafruit_Sensor.h> #include <Adafruit_BMP085_U.h> #include <SPI.h>
#include <LoRa.h>

// Timing
unsigned long previousMillis = 0;
const unsigned long interval = 100; // 100ms = 10 Hz sending

// Pin definitions #define TRIG_PIN 6
#define ECHO_PIN 7
#define DHTPIN 5
#define DHTTYPE DHT11 // DHT11 or DHT22
#define RAIN_PIN 4
#define FLOW_SENSOR_PIN 3 // Flow sensor pin const float
PRESSURE_THRESHOLD = 20;
// Flow sensor variables
volatile int flowPulseCount = 0; unsigned long lastFlowCalcTime = 0; float flowRate
= 0.0; // in L/min

// Sensor objects
Adafruit_BMP085_Unified bmp = Adafruit_BMP085_Unified(10085);
LiquidCrystal_I2C lcd(0x27, 16, 2);
DHT dht(DHTPIN, DHTTYPE);

// Flow sensor interrupt void flowPulse() { flowPulseCount++;
```

```
}

void setup() {
  // Serial Serial.begin(9600);

  // Ultrasonic pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);

  // Rain sensor pinMode(RAIN_PIN, INPUT);

  // Flow sensor
  pinMode(FLOW_SENSOR_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(FLOW_SENSOR_PIN), flowPulse, RISING);
  // DHT dht.begin();

  // BMP180
  if (!bmp.begin()) {
    Serial.println(F("Could not find BMP180 sensor!")); while (1);
  }

  // LCD lcd.init(); lcd.backlight();
  lcd.setCursor(0, 0); lcd.print("Flood Monitoring"); delay(2000);
  lcd.clear();

  // LoRa
  if (!LoRa.begin(433E6)) { lcd.clear(); lcd.setCursor(0, 0); lcd.print("LoRa Failed");
    delay(1000);
    while (1);
  }
  lcd.clear(); lcd.setCursor(0, 0); lcd.print("LoRa Sender"); delay(1000);
}
```

```
void loop() {
  unsigned long currentMillis = millis();

  // --- Read Flow Sensor ---
  unsigned long flowElapsed = currentMillis - lastFlowCalcTime; if (flowElapsed >=
  1000) { // update every 1 second noInterrupts();
  int count = flowPulseCount; flowPulseCount = 0; interrupts();

  // YF-S201: 450 pulses per liter
  flowRate = (count / 450.0) * (60000.0 / flowElapsed); // L/min

  lastFlowCalcTime = currentMillis;
}

// --- Sensor read interval ---
if (currentMillis - previousMillis >= interval) { previousMillis = currentMillis;

//   Ultrasonic   digitalWrite(TRIG_PIN,   LOW);   delayMicroseconds(2);
digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10); digitalWrite(TRIG_PIN,
LOW);
long duration = pulseIn(ECHO_PIN, HIGH); float distance = duration * 0.034 / 2;

// BMP180 pressure sensors_event_t event; bmp.getEvent(&event);
float pressure = event.pressure / 100.0F;

// DHT
float humidity = dht.readHumidity();
float temperature = dht.readTemperature();
// Rain sensor
int rainValue = digitalRead(RAIN_PIN);
```

```
// Display on LCD lcd.clear(); lcd.setCursor(0, 0); lcd.print("Dist:"); lcd.print(distance,
1); lcd.print("cm");

lcd.setCursor(0, 1); lcd.print("Flow:"); lcd.print(flowRate, 2); lcd.print("L/min");
delay(2000);

lcd.clear(); lcd.setCursor(0, 0); lcd.print("Temp:"); lcd.print(temperature, 1);
lcd.print("C");

lcd.setCursor(0, 1); lcd.print("Hum:"); lcd.print(humidity, 0); lcd.print("%");
delay(2000);

lcd.clear(); lcd.setCursor(0, 0); lcd.print("Rain:"); lcd.print(rainValue); lcd.setCursor(0,
1); lcd.print("Pressure:"); lcd.print(pressure);
delay(2000);

// --- Send via Serial & LoRa ---
String Values = "a" + String(distance) + "b" + String(temperature) + "c" +
String(rainValue) + "d" + String(flowRate) +
"e" + String(humidity) + "f" + String(pressure) + "g";

Serial.println(Values); lcd.clear(); lcd.setCursor(0, 0); lcd.print(Values); delay(2000);
LoRa.beginPacket(); LoRa.print(Values); LoRa.endPacket();
}
}
```

5.3 SYSTEM TESTING

5.3.1 Software Testing

Creating Thingspeak Account

The first thing you need to do is to create an account with ThingSpeak. Since the collaboration with MATLAB, you can use your MathWorks credentials to login to

ThingSpeak using the Sign In link from this page: **THINGSPEAK**. After logging in, you need to create a new channel for the data to be stored. For this go to Channels→My Channels and click on New Channel.

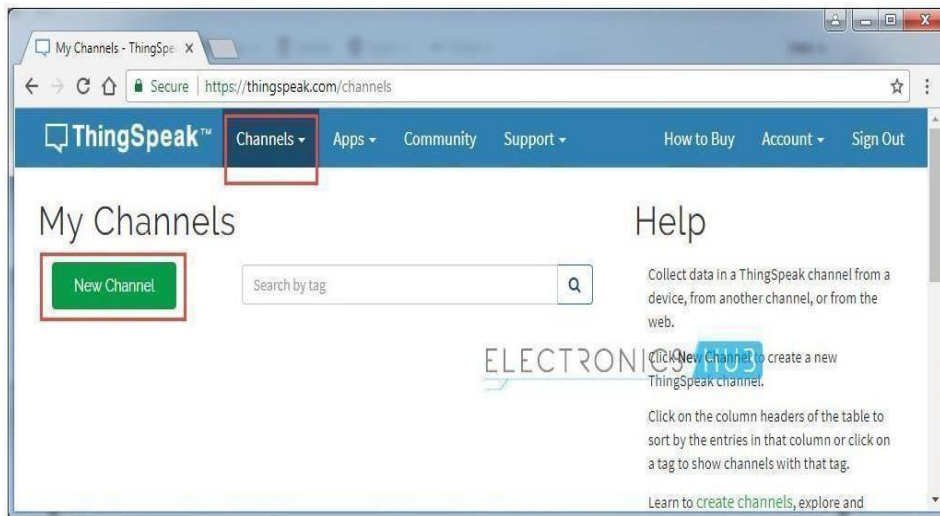


Figure 5.21: Creating new Channel

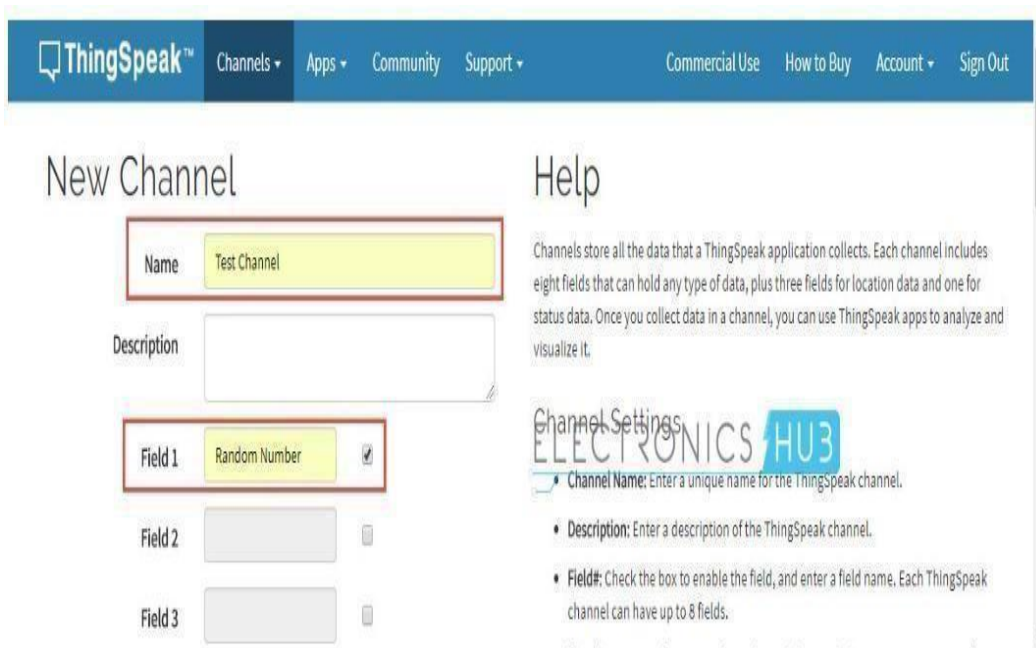


Figure 5.22: Providing the data

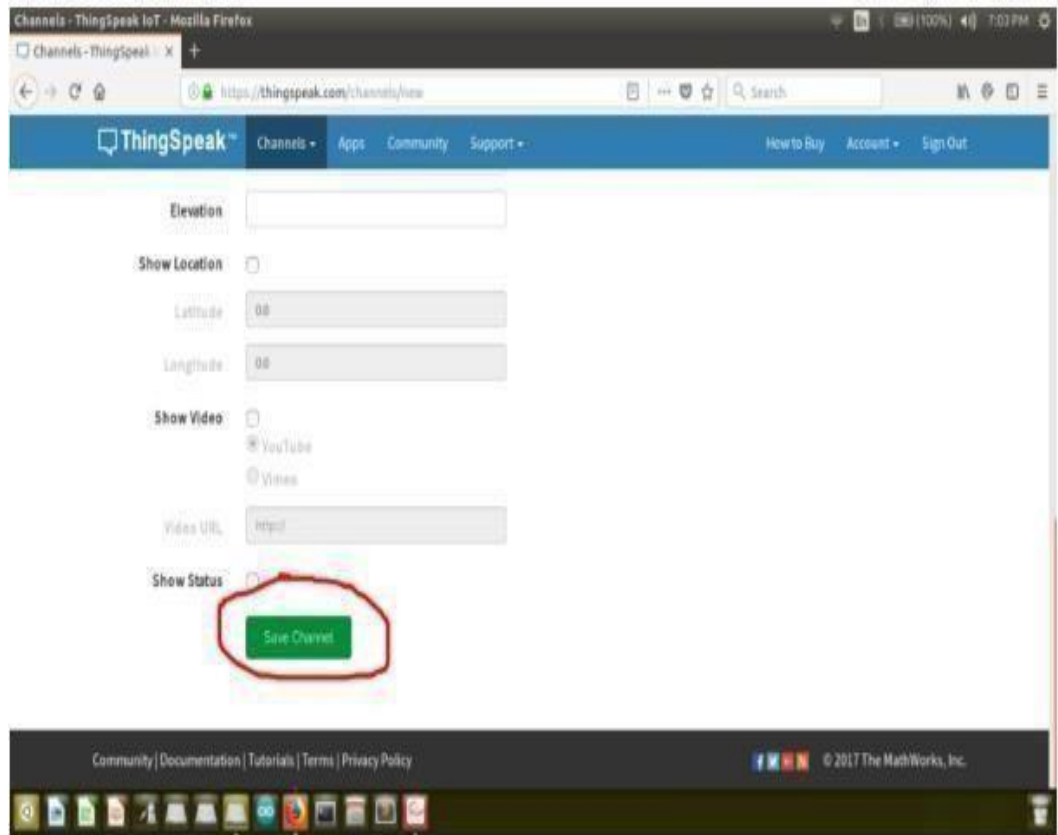


Figure 5.23: Saving contents

Getting API Key

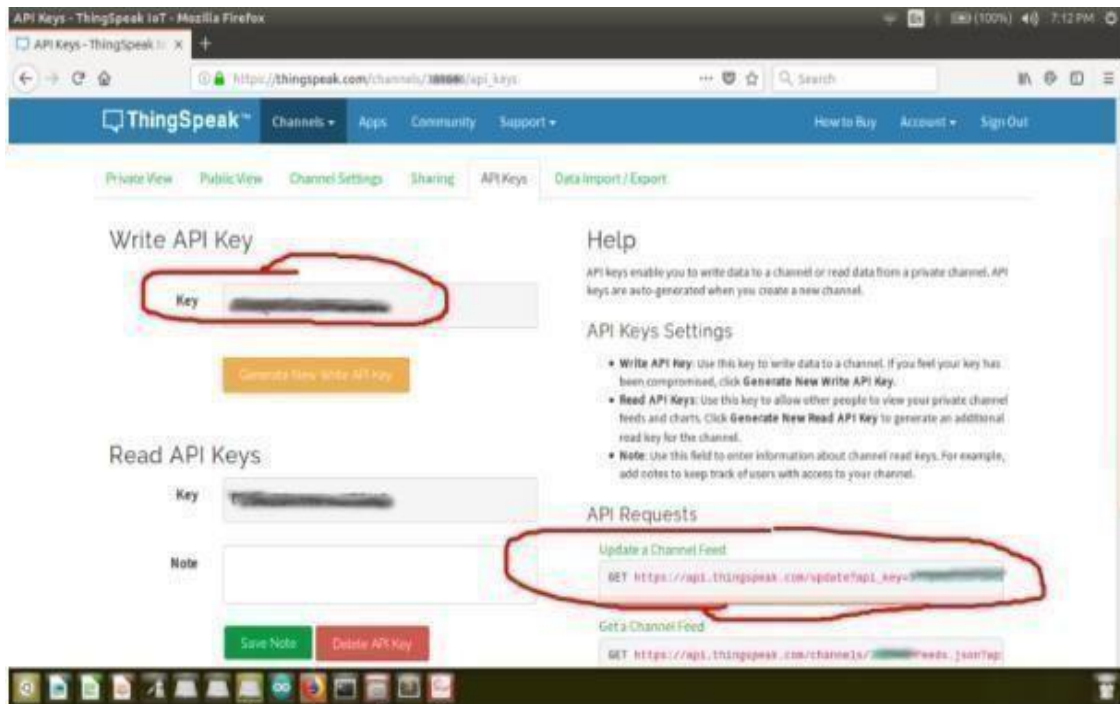


Figure 5.24: Click on API Key Tab

```
#include <ESP8266WiFi.h>
#include "secrets.h"
#include "ThingSpeak.h" // always include thingspeak header file after other header files and custom macros
#include<LiquidCrystal.h>

LiquidCrystal lcd(D2,D3,D5,D6,D7,D8);

char ssid[] = SECRET_SSID; // your network SSID (name)
char pass[] = SECRET_PASS; // your network password
int keyIndex = 0; // your network key Index number (needed only for WEP)
WiFiClient client;

unsigned long myChannelNumber = SECRET_CH_ID;
const char * myWriteAPIKey = SECRET_WRITE_APIKEY;
```

```
int x = ThingSpeak.writeField(myChannelNumber, 1, batteryValue, myWriteAPIKey);
if(x == 200){
    Serial.println("Channel update successful.");
}
else{
    Serial.println("Problem updating channel. HTTP error code " + String(x));
}
```

Figure 5.24: Code for NodeMCU

Check Data on ThingSpeak Server

Open Your ThingSpeak Account and Click on Private View of Your Chart

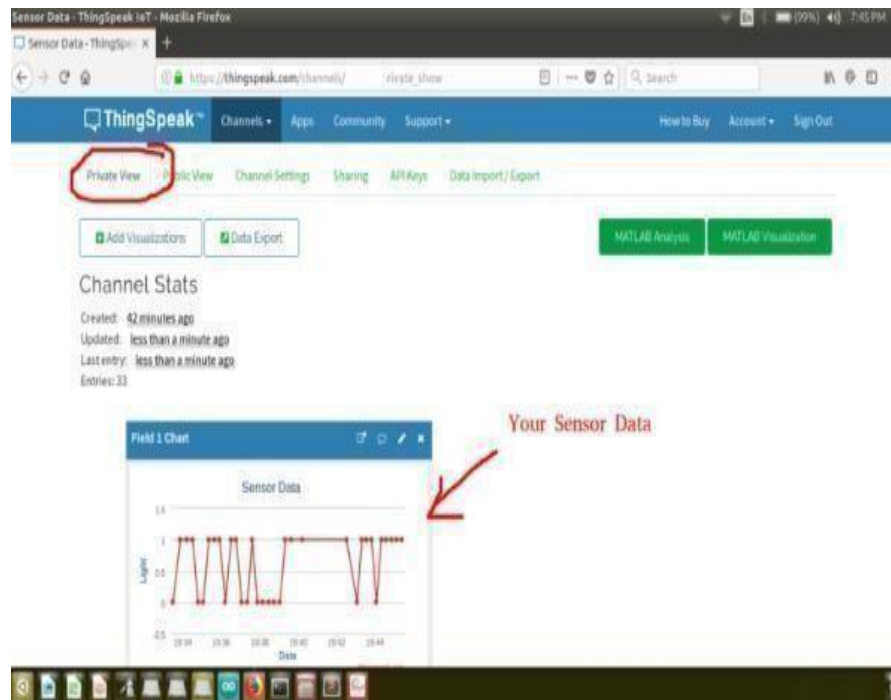


Figure 5.25: uploading data into thingspeak

5.3.2 Types of Tests

5.3.2.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

5.2.3.2 Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components

5.3.2.3 Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

5.3.3 Test Cases

Table 5.1: Test Case 1 –flood Detection and Alert Generation

S. No	Test Case Id	Flood alert_Alert
1	Precondition	The system is powered on and sensors are active
2	Description	Detects an flood using sensors and sends alert
3	Test Steps	-Simulate a crash or tilt using MEMS sensor -Check if GPS retrieves location -Verify if GSM sends alert to emergency contacts
4	Expected Output	Flood detected GPS location retrieved, alert sent via GSM
5	Actual Output	-Flood detected detected, GPS location retrieved, -alert sent via GSM
6	Status	Pass
7	Remarks	-

Description: The system detects flood conditions using sensors and sends alerts.

Test Steps: A flood is simulated using sensors; the system checks GPS location and sends an alert via GSM to emergency contacts.

Expected Output: Flood is detected, GPS location is obtained, and an alert is sent.

Actual Output: The system successfully detects the flood, retrieves GPS location, and sends the alert.

Table 5.2: Test Case 2 – flood Detection and Warning

S. No	Test Case Id	flood_Warning
1	Precondition	The system is turned on and sensors is active
2	Description	Detects pressure and issues a warning
3	Test Steps	-Blow air near the sensor Check if alert is triggered -Confirm warning is shown on LCD
4	Expected Output	-flood detected, warning issued on speaker and LCD
5	Actual Output	Flood detected, warning issued on speaker and LCD
6	Status	pass
7	Remarks	-

Description: The system detects pressure changes and generates a warning.

Test Steps: Air is blown near the sensor to simulate pressure; then it checks if the alert is triggered and displayed on the LCD.

Expected Output: Flood is detected and a warning is issued through both the speaker and LCD.

Actual Output: The system successfully detects the flood and gives a warning on the speaker and LCD.

Status: Pass (system is working correctly).

Table 5.3: Test Case 3 – buzzer Functionality

S. No	Test Case Id	Emergency_Button
1	Precondition	System is powered on and communication modules are connected
2	Description	Manual alert is triggered using the buzzer
3	Test Steps	Confirm alert is sent via GSM to emergency contacts
4	Expected Output	Manual alert sent with GPS location to emergency services
5	Actual Output	Manual alert sent with GPS location to emergency services
6	Status	pass
7	Remarks	–

Description: A manual alert is triggered using a buzzer (emergency button).

Test Steps: Press the emergency button and check whether an alert is sent via GSM to emergency contacts.

Expected Output: A manual alert along with GPS location is sent to emergency services.

Actual Output: The system successfully sends the alert with GPS location to emergency services.

Status: (Implied Pass, since expected and actual outputs match).

CHAPTER 6

RESULTS

6.1 EXECUTION PROCEDURE

1. **Hardware Setup:** Assemble Arduino UNO with sensors (rain, ultrasonic, BMP180, flow, DHT11), LoRa SX1278, NodeMCU, GPS, GSM, buzzer, LED, LCD, and solar-battery power system.
2. **Firmware Development:** Write Arduino sketches to read sensor data, handle LoRa transmission, and manage alerts (buzzer/LED/LCD).
3. **Python AI Integration:** Develop Python scripts (machine-learning models) for data processing and flood prediction; interface Python with NodeMCU for cloud upload and analytics.
4. **Communication Configuration:** Set up LoRaWAN and Wi-Fi (NodeMCU) for data transmission; configure GSM for SMS alerts.
5. **System Testing:** Run real-time sensor data acquisition, verify AI predictions, and validate alert mechanisms (buzzer, SMS, cloud display).
6. **Deployment:** Install the solar-powered unit in the target environment; ensure continuous monitoring and early-warning functionality.
7. **Maintenance:** Monitor system health, update AI models, and optimize power consumption for long-term operation.

6.1 SCREEN SHOTS

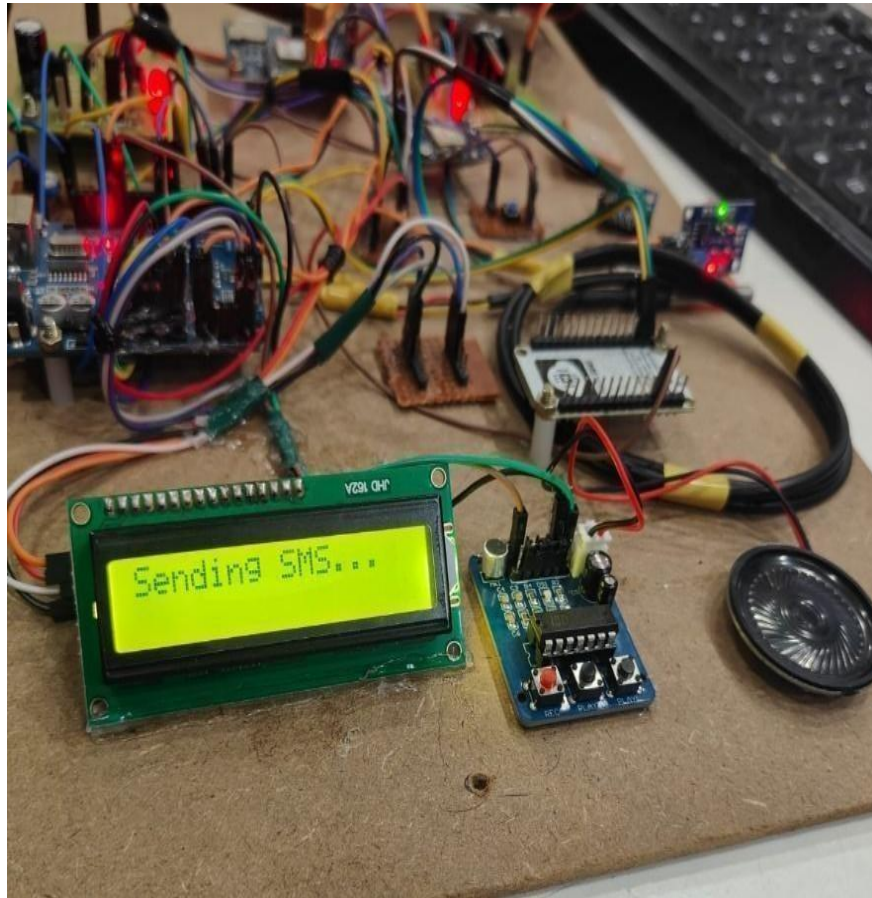


Figure 6.1: Sending Message

Displays the IoT-based Smart flood detection with sensors, a microcontroller, and an LCD, actively sending SMS alerts . A green LCD display with a yellow backlight showing the message “Sending SMS...”, indicating the system is transmitting an alert via SMS. A blue microcontroller board (likely an Arduino or similar) connected to the LCD and other peripherals.. A GSM module (black module with antenna) used for sending the SMS alerts. A small black buzzer or speaker next to the microcontroller for audible alerts. Various jumper wires connecting the components on a workspace surface..

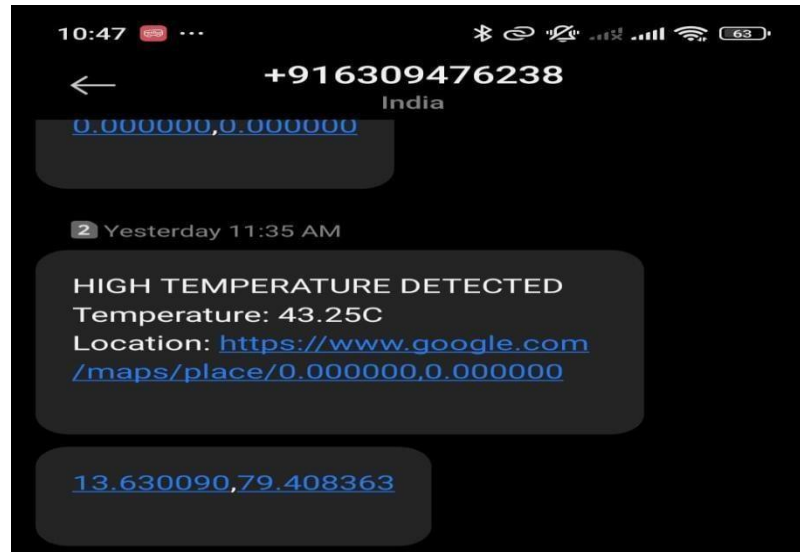


Figure 6.2:High Temperature Detected

The System detects the high temperature via SMS and real-time GPS coordinates.

- HIGH TEMPERATURE DETECTED
- Temperature: 43.25°C
- Location: a Google Maps link with coordinates 0.000000,0.000000 (placeholder).
- The actual GPS coordinates are shown below as 13.630090, 79.408363.

The setup demonstrates an IoT (LoRaWAN) system that sends an alert SMS when an abnormal temperature is sensed, including the exact location via GPS coordinates.



Figure 6.3:Displaying Latitude and Longitude

The LCD screen shows real-time latitude and longitude.

1. Lat: 13.427487 – the latitude coordinate.
2. Lon: 79.575157 – the longitude coordinate.

The setup illustrates how a GPS or navigation module outputs geographic position values (latitude and longitude) on the display.

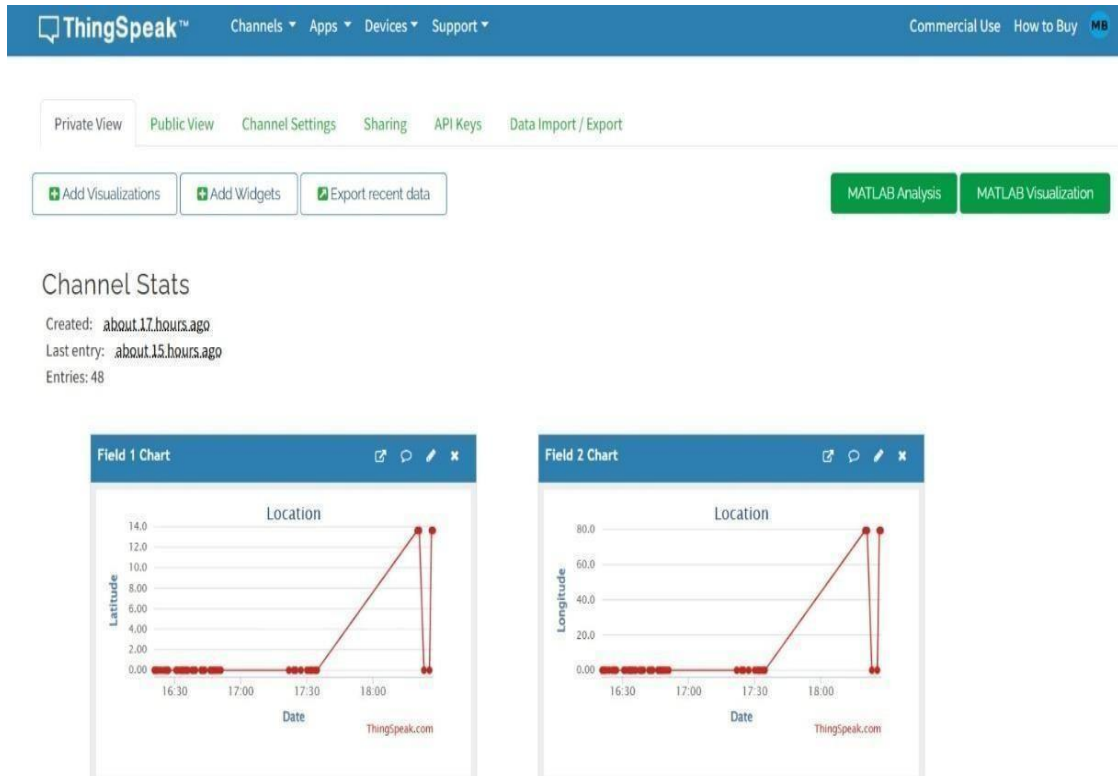


Figure 6.4: Visualization of real-time flood detection using ThinkSpeak

flood detection data is logged and visualized on ThingSpeak, showing movement patterns over time using latitude and longitude graphs. This screenshot represents a mixed or cluttered display of mobile UI and background material, where multiple layers (document, charts, and app interface) are visible at once, making the content partially unreadable and unstructured.

Cleaned Data: a19.04b26.70c1d0.00e63.00f18.37g

distance_val: 19.04

temp_val: 26.7

rain_Val: 1.0

flow_Val: 0.0

humidity: 63.0

pressure: 18.37

Predictions:

U Prediction: 1

V Prediction: 1

W Prediction: 1

X Prediction: 1

Feature 1: Normal ☒

Feature 2: Normal ☒

Feature 3: Normal ☒

Feature 4: Normal ☒

Sent to serial: tlulvwlx

Cycle completed ☒

Figure 6.5:output screenshots

The system detects pressure,atmosphere,rain ,humidity and sends an alert with the gsm coordinates. The screenshot displays processed sensor data and prediction results generated by the system. At the top, cleaned data values are shown, representing inputs from multiple sensors.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

7.1 CONCLUSION

The AI-Driven LoRaWAN-Enabled Solar-Powered Multi-Sensor Framework provides a smart, reliable, and energy-efficient solution for proactive flood prediction and early warning. By integrating multiple sensors, IoT connectivity, and AI-based predictive analysis, the system continuously monitors environmental and water conditions, enabling timely alerts through GSM, buzzer, LED, LCD, and IoT platforms. Its solar-powered design ensures uninterrupted operation, even in remote or power-deficient areas. This intelligent framework enhances disaster preparedness, reduces response time, and minimizes the impact of floods on communities, demonstrating a scalable and sustainable approach to flood management.

7.2 FUTURE ENHANCEMENT

Enhancing the flood prediction and early warning system by incorporating additional environmental sensors significantly improves the depth and reliability of real-time data collection, enabling the system to monitor a wider range of parameters such as soil moisture, atmospheric pressure, rainfall intensity, humidity, and water flow dynamics with greater precision, which in turn strengthens the system's ability to detect subtle environmental changes that precede flood events. Improving the AI model accuracy involves leveraging advanced machine learning techniques, larger and more diverse datasets, and continuous model training to reduce prediction errors and enhance the system's capability to identify complex patterns and correlations between environmental factors, ultimately resulting in more timely and accurate flood forecasts.

Integrating a mobile application for alerts transforms the system into a user-centric solution by ensuring that real-time warnings, risk levels, and safety instructions are instantly delivered to individuals, communities, and authorities, thereby increasing preparedness and enabling faster response during critical situations. The inclusion of satellite and weather data further elevates the system's predictive power by providing large-scale and high-resolution insights into cloud movement, rainfall distribution, storm development, and geographical changes, which complements ground-level sensor data and allows for more comprehensive and accurate forecasting across different regions.

REFERENCES

- [1] M. Sathiyamoorthy and P. Subramanian, "AI-Driven Early Warning System for Predicting and Mitigating Urban Flooding: A Comparative Analysis of Real-Time Data and Predictive Algorithms," *2025 2nd International Conference on Computing and Data Science (ICCDS)*, Chennai, India, 2025, pp. 1-6, doi: 10.1109/ICCDS64403.2025.11209450.
- [2] T. Duan, P. Li, P. Huang, and W. Wei, "Fast urban flood modeling informing response decisions: Model development and future perspectives," *Advanced Engineering Informatics*, vol. 70, p. 104152, Dec. 2025, doi: 10.1016/j.aei.2025.104152.
- [3] S. M and S. P, "AI-Driven Early Warning System for Predicting and Mitigating Urban Flooding: A Comparative Analysis of Real-Time Data and Predictive Algorithms for Enhanced Flood Management," *2025 2nd International Conference on Computing and Data Science (ICCDS)*, Chennai, India, 2025, pp. 1-6, doi: 10.1109/ICCDS64403.2025.11209748.
- [4] L.-C. Chang, M.-T. Yang, and F.-J. Chang, "Flood resilience through hybrid deep learning: Advanced forecasting for Taipei's urban drainage system," *Journal of Environmental Management*, vol. 379, p. 124835, Mar. 2025, doi: 10.1016/j.jenvman.2025.124835.
- [5] T. Q. Dang et al., "Integrating Intelligent Hydro-informatics into an effective Early Warning System for riskinformed urban flood management," *Environmental Modelling & Software*, vol. 183, p. 106246, Oct. 2024, doi: 10.1016/j.envsoft.2024.106246.
- [6] C. M. Ranieri et al., "Water level identification with laser sensors, inertial units, and machine learning," *Engineering Applications of Artificial Intelligence*, vol. 127, p. 107235, Oct. 2023, doi: 10.1016/j.engappai.2023.107235.
- [7] P. M. Rasalkar and A. R. Surve, "Evolution of Flood Prediction Methods: Hydrological, Statistical, Machine Learning and Deep Learning Approaches (2020–2025)," *2025 International Conference on Future Technologies (ICFT)*, Sangli, India, 2025, pp. 1-6, doi: 10.1109/ICFT66708.2025.11336611.
- [8] P. Charan, M. M. Siddiqui, V. Yadav, C. Pathak, Y. Narayan and Z. H. Khan, "AI-Enhanced Early Warning Systems for Natural Disaster Detection and Mitigation using Wireless Sensor Networks," *2024 Second International Conference Computational and Characterization Techniques in Engineering & Sciences (IC3TES)*, Lucknow, India, 2024, pp. 1-6, doi: 10.1109/IC3TES62412.2024.10877562.

- [9] D. Yu, Q. Tao, Q. Liu, Y. Jin, Y. Sun, and P. Fu, "Lifecycle management of urban renewal enabled by Internet of Things: Development, application, and challenges," *Results in Engineering*, vol. 27, p. 105706, Jun. 2025, doi: 10.1016/j.rineng.2025.105706.
- [10] M. Dash, C. Parida, M. Pandey, A. Dash, G. Bhuyan and S. Darshana, "AI-Driven Climate Disaster Prediction Using ConvLSTM and GANs," *2025 International Conference on Artificial intelligence and Emerging Technologies (ICAIET)*, Bhubaneswar, India, 2025, pp. 1-6, doi: 10.1109/ICAIET65052.2025.11211377.
- [11] S. B. Bhonde, H. S. Chattar, S. S. Gawande, K. S. Fulsoundar and H. S. Bharitkar, "Modeling and Forecasting River Flood Inundation Using Machine Learning," *2025 9th International Conference on Computing, Communication, Control and Automation (ICCCBEA)*, Pune, India, 2025, pp. 1-6, doi: 10.1109/ICCUBEA65967.2025.11283912.
- [12] J. Ali et al., "A deep dive into cybersecurity solutions for AI-driven IoT-enabled smart cities in advanced communication networks," *Computer Communications*, vol. 229, p. 108000, Nov. 2024, doi: 10.1016/j.comcom.2024.108000.

